Akiba SACCO USSD Emulator

How every part works, explained simply: from a screen written as data to the pixels on the phone, plus the TypeScript and front-end ideas behind it

Project folder: `~/WebstormProjects/akiba` · second edition, for the restructured code (`src/ussd`, `src/phone`, `src/log`)

Prepared for Stephen Oloo (Nemiceps) · 12 November 2026

Contents

How to read this guide

Part 0 — Front-end basics in plain words

- 0.1 The three languages of a web page
- 0.2 What the browser does when you open index.html
- 0.3 Events: how clicks reach the code
- 0.4 One thread, and how "waiting" works
- 0.5 Where a web page can keep data
- 0.6 Three DOM habits you will see everywhere

Part 1 — The big picture

- 1.1 What the app is
- 1.2 USSD in five facts
- 1.3 The folders
- 1.4 The three layers
- 1.5 The single most important idea
- 1.6 Build and run

Part 2 — The pipeline: from a screen definition to pixels

- 2.1 The whole journey on one page
- 2.2 Step ① — Writing the definition
- 2.3 Step ② — Compiling
- 2.4 Step ③ — Joining features into one registry
- 2.5 Step ④ — The session: the state the rules are applied to
- 2.6 Step ⑤ — renderSession(): rules + state → text
- 2.7 Step ⑥ — phone.view(): adding what only the handset knows
- 2.8 Step ⑦ — LcdDisplay.render(): View → DOM
- 2.9 Step ⑧ — The browser turns the DOM into pixels
- 2.10 The same pipeline with different inputs
- 2.11 Every intermediate value: why it exists, and could it be removed?

Part 3 — The reply pipeline: from a key press to a new session state

- 3.1 The whole journey
- 3.2 Typing digits: only the phone changes
- 3.3 Pressing Send: the engine decides immediately
- 3.4 Inside engine.reply()
- 3.5 handle(): what does the input mean?
- 3.6 Logging
- 3.7 Back on the phone: waiting, then delivering
- 3.8 The three similar result objects

Part 4 — TypeScript, explained with this code

- 4.1 What TypeScript is (and is not)
- 4.2 Declaring things: const, let, arrow functions
- 4.3 Annotations
- 4.4 type and interface

4.5 Literal types and unions (TypeScript's enums)
4.6 Optional fields, undefined and null
4.7 Null-handling operators
4.8 Discriminated unions — the key to this codebase
4.9 Narrowing and type guards
4.10 Utility types
4.11 keyof, typeof, and indexed access
4.12 as const and satisfies
4.13 Generics
4.14 A subtle one: method bivariance (why SCREEN_TYPES compiles)
4.15 Computed keys and assertions
4.16 unknown instead of any
4.17 Classes
4.18 Modules: import, export, import type, .js
4.19 Asynchronous code
4.20 Everyday syntax
4.21 What tsconfig.json switches on

Part 5 — File-by-file reference

5.0 What changed from the previous version
5.1 Tooling and configuration files
5.2 Shared files at the top of src/
5.3 src/ussd/types.ts — every data shape
5.4 src/ussd/screens/ — the menu as data
5.5 src/ussd/screenTypes.ts — how each kind of screen behaves
5.6 src/ussd/text.ts — words and sizes
5.7 src/ussd/engine.ts — sessions
5.8 src/ussd/checks.ts — startup checks
5.9 src/phone/phone.ts — the handset
5.10 src/phone/display.ts — the LCD
5.11 src/phone/main.ts — entry point of index.html
5.12 src/log/logRepo.ts — storing the session log
5.13 src/log/main.ts — the session-log page
5.14 src/log/dataTable.ts — a reusable data-table component
5.15 HTML pages
5.16 Stylesheets
5.17 Tests

Part 6 — Design choices, alternatives, weak spots, questions

6.1 Screens as data, versus other ways to build a menu
6.2 SCREEN_TYPES lookup table, versus switch, versus classes
6.3 Functions inside screen definitions, versus names
6.4 The Reply object, versus letting screen types change the session
6.5 A mutable session in the engine, versus immutable state in the phone
6.6 Screen IDs as plain strings
6.7 Running the checks in the browser at startup

6.8 Text as one string with \n, and hand-placed line breaks
6.9 Validators that change the account
6.10 Where things run: browser, engine, phone
6.11 Framework-free, versus React
6.12 Logging choices
6.13 Known weak spots (raise them yourself)
6.14 Question bank
6.15 Cheat sheet

How to read this guide

This guide assumes you can program (Java, Go, SQL) but are new to **front-end development** and **TypeScript**. Nothing is assumed about browsers.

It is organised so you can stop at any point and still understand the project:

Part	What you get	Read it if...
0. Front-end basics	What a browser does with HTML, CSS and JavaScript; events; timers; the DOM	You have never built a web page
1. The big picture	What the app is, the folders, the three layers	Always
2. The pipeline ★	One screen followed from its definition in `screens/payments.ts` all the way to pixels on the LCD, with the **real value at every step**, why each step exists, whether it is necessary, and other ways it could have been done	Always — this is the heart of the guide
3. The reply pipeline	What happens when you press a key and send a reply, step by step	Always
4. TypeScript, explained with this code	Every TypeScript feature the project uses	Before reading the code
5. File-by-file reference	Every file and every function	When you open a file
6. Design choices, alternatives, weak spots, questions	Pros and cons of each big decision, what a reviewer might criticise, and answers to likely questions	Before any review or interview

All example values in Parts 2 and 3 were produced by **running your current code** (tests: 46 of 46 pass), not written by hand.

> **How to study.** Open WebStorm next to this PDF. For every step in Part 2, find the function named, put a breakpoint on it, run `yarn start`, open the page, and watch the values in the debugger match the ones printed here. Doing this once for one screen will teach you more than reading the whole guide twice.

Part 0 — Front-end basics in plain words

You only need a handful of ideas to understand this project.

0.1 The three languages of a web page

Language	Its job	In this project
HTML	The *structure*: which boxes and buttons exist	`index.html` (the phone), `log.html` (the log page)
CSS	The *look*: sizes, colours, fonts, what is visible	`styles/emulator.css`, `styles/log.css`, `styles/data-table.css`
JavaScript	The *behaviour*: what happens when you click	Written as TypeScript in `src/`, compiled to `dist/`

Browsers only run JavaScript. **TypeScript is JavaScript with types added**; the compiler `tsc` checks the types and then deletes them, producing plain JavaScript in `dist/`.

0.2 What the browser does when you open `index.html`

1. It **reads the HTML** and builds a tree of objects in memory called the **DOM** (Document Object Model). Every tag becomes an object: the `<button data-key="5">` becomes an `HTMLButtonElement` object you can read and change from JavaScript.
2. It **applies the CSS** to decide each box's size, colour and position, then **paints** the result on screen.
3. It **runs the script** `<script type="module" src="dist/phone/main.js">`. Because it is a *module*, it runs after the HTML has been read, and its `import` lines make the browser fetch the other `.js` files it needs.

Changing the DOM from JavaScript (for example `element.textContent = 'Hello'`) makes the browser repaint that part of the page automatically. **That is how the phone screen updates**: the code never draws pixels itself; it changes text in DOM elements and the browser draws them.

0.3 Events: how clicks reach the code

The browser tells your code about things that happen through **events**: `click`, `keydown`, `change`, `visibilitychange`, and so on. You register a function to run when an event happens:

```
button.addEventListener('click', () => phone.press('5'));
```

This is like registering a handler on a route in Undertow, except the "request" is a click.

0.4 One thread, and how "waiting" works

JavaScript in a page runs on **one thread**. There is no `Thread.sleep`. To do something later you schedule it:

```
setTimeout(() => this.deliver(), 600);   // run deliver() in about 600 ms
```

`setTimeout` returns immediately; the browser calls the function later when the thread is free. The phone uses this to *pretend* the network takes 600 ms to answer. While the timer is pending the page stays responsive: you can still press keys.

0.5 Where a web page can keep data

Storage	Kind	Used for
Variables in memory	Lost on reload	The account, the session, the phone state
`localStorage`	Small, text only, synchronous	The chosen language, the log table's view settings
`IndexedDB`	A real database in the browser: objects, indexes, transactions, asynchronous	The session log

0.6 Three DOM habits you will see everywhere

- **`document.querySelector('[data-lcd="root"]')`** finds the first element matching a CSS selector. This project finds elements by `data-*` attributes, not CSS classes, so restyling never breaks the code.
- **`element.textContent = text`** sets text safely. Anything in `text` is shown as characters, never interpreted as HTML. (The opposite, `innerHTML`, would run injected tags — a security risk called XSS.)
- **`element.dataset.view = 'session'`** sets the attribute `data-view="session"`. CSS can then react to it, for example to show only the session panel.

That is all the front-end you need. Everything else in the project is ordinary programming.

Part 1 — The big picture

1.1 What the app is

Akiba SACCO is a savings co-operative whose members use basic phones. They dial `*144#` and move through text menus to check their balance, withdraw money, buy airtime and see a mini statement. The brief (`docs/Task - USSD on Web.pdf`) asks for a **browser emulator** of that phone so staff can test menus, train agents and check that text fits. The hard rules from the brief:

Rule	Where the code enforces it
Screen is exactly **160 × 128 px**, no scrolling	CSS fixes the size; `ussd/text.ts` counts rows; `phone/display.ts` measures the real DOM
Input only from the **on-screen keypad**	`phone/main.ts` turns buttons (and keyboard shortcuts) into `phone.press(key)`
English and Swahili on every screen	`messages.ts`; screens hold message IDs, never text
Zero dependencies, **TypeScript** for logic	No framework; only `typescript` as a build tool

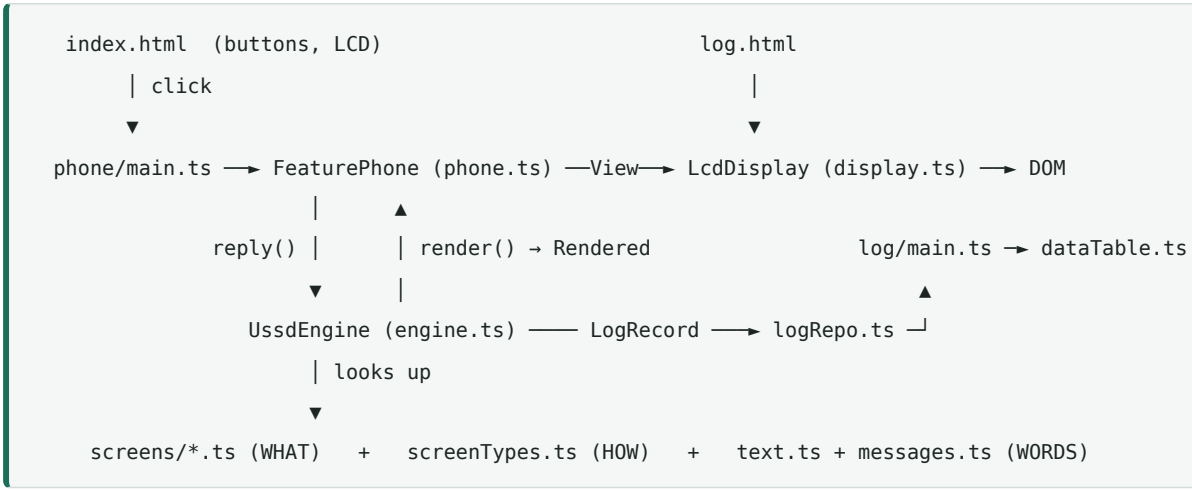
1.2 USSD in five facts

1. **A live session.** You dial, the network sends a screen, you reply with a number, it sends the next screen.
2. **CON or END.** Every network answer either *continues* (show text and wait) or *ends* (show text and close).
3. **Size limit.** A screen can carry at most **182 GSM-7 characters**, or only **80** if any character needs Unicode (UCS-2).
4. **Timeouts.** The network drops a session after inactivity (here 30 s) or a maximum duration (here 180 s).
5. **Conventions.** `0` = back, `00` = main menu, `98` = more (next page).

1.3 The folders

```
src/
├── config.ts           numbers you might tune: *144#, timeouts, delays, screen grid
├── messages.ts         every piece of text, in English and Swahili
├── mockData.ts         a fake account (KES 12,450.00, PIN 1234) and three banks
├── ussd/               ── THE NETWORK SIDE: knows nothing about HTML ──
│   ├── types.ts          every data shape (Session, Screen, Msg, …) — read this first
│   ├── screens/          THE MENU, AS DATA
│   │   ├── index.ts        joins the feature files into one registry
│   │   ├── login.ts        PIN screen (+ its guard and validator)
│   │   ├── menu.ts         main menu, balance
│   │   ├── payments.ts     withdraw, airtime, phone, bank, amount, confirm (+ actions)
│   │   └── statement.ts    mini statement
│   ├── screenTypes.ts    HOW each kind of screen is drawn and what replies it accepts
│   ├── text.ts           message → text ({{placeholders}}), size measuring
│   ├── engine.ts         sessions: dial, reply, back/home, timeouts, logging
│   └── checks.ts         startup checks: wiring, translations, every screen fits
├── phone/              ── THE HANDSET SIDE ──
│   ├── phone.ts          the handset state machine (idle, dialling, waiting, …)
│   ├── display.ts        writes a View into the LCD's HTML elements
│   └── main.ts           entry point of index.html: builds everything, wires buttons
└── log/                ── THE LOG PAGE ──
    ├── logRepo.ts        stores log records (IndexedDB in the browser, memory in tests)
    ├── main.ts           entry point of log.html
    └── dataTable.ts      reusable sortable/filterable table (ported from Survey Pad)
```

1.4 The three layers



Layer	Files	Plain meaning	Java analogy
UI	`phone/main.ts`, `phone/display.ts`, `log/*`	Touches HTML; turns clicks into calls and data into text on screen	Handler / controller

Layer	Files	Plain meaning	Java analogy
Device	`phone/phone.ts`	What the handset is doing right now; fakes network delay	A small stateful service
Network / domain	`ussd/*`	Sessions, menus, rules; pure logic, no HTML, no timers	Service + rules
Storage	`log/logRepo.ts`	Saves the log	Repository

1.5 The single most important idea

The menu is data, not code. A screen is a plain object:

```
withdraw: {
    type: 'menu',
    prompt: 'withdraw.prompt',
    options: {
        1: {label: 'withdraw.mobile', next: 'chooseNumber',
            set: {service: 'withdraw', channel: 'mobile', fee: 30}},
        2: {label: 'withdraw.bank', next: 'bankList',
            set: {service: 'withdraw', channel: 'bank', fee: 100}},
    },
},
```

It says *what* the screen is (a menu, its title, its two options and where they lead). It does not say *how* to draw a menu or how to react to `2`. That "how" lives once, in `screenTypes.ts`, for every menu. The engine glues the two together. So:

- **Adding a screen** = writing one object like the one above.
- **Adding a new kind of screen** = describing it in `types.ts` and adding one entry to `SCREEN_TYPES` in `screenTypes.ts`.
- **The engine never changes** for either.

Part 2 follows this exact idea from the object above to the pixels.

1.6 Build and run

```
src/**/*.ts ──tsc──► dist/**/*.js ──scripts/build-site.mjs──► public/ ──► Vercel
                          └──── scripts/serve.mjs ──► http://localhost:5173
```

Command	Does
`yarn start`	Compile, then serve locally on port 5173
`yarn watch`	Recompile on every save (run alongside `yarn serve`)
`yarn test`	Compile, then run the 46 tests in `tests/`
`yarn build`	Compile, then copy the site into `public/` for deployment

Part 2 — The pipeline: from a screen definition to pixels

We follow **one screen**, the withdrawal confirmation, from the moment you write it until it appears on the LCD. The member has: logged in, chosen *2. Withdraw Money*, *1. Mobile Money*, *1. My Number*, and typed *5000*.

2.1 The whole journey on one page

Each box is a real intermediate value. Each arrow is a function. The rest of this part explains every box and every arrow.

```
① DEFINITION  (screens/payments.ts — written by you)
   confirm: {type:'menu', prompt:{by:'service', cases:{withdraw:'confirm.withdraw', …}},
             options:{1:{label:'confirm.yes', action:commitTransaction}, 2:{…}},
             inline:true, nav:false}
      │  tsc: check types, then erase them                          (compile time)
      ▼
② COMPILED OBJECT  (dist/ussd/screens/payments.js) — same object, no types
      │  screens/index.ts merges all feature files
      ▼
③ REGISTRY  {entry:'enterPin', home:'main', screens:{enterPin, main, …, confirm, …}}
      │  checks.ts assertRegistry(): wiring + translations + sizes   (page load)
      ▼
④ SESSION  (state — changes on every reply)
   {screen:'confirm', page:0, open:true, error:undefined,
    data:{recipient:'0712345678', service:'withdraw', channel:'mobile',
          fee:30, amount:5000},
    history:['main','withdraw','chooseNumber','enterAmount','confirm']}
      │  engine.render(session,'en')  →  renderSession()
      │    • definition = registry.screens['confirm']
      │    • behaviour  = SCREEN_TYPES['menu']
      │    • title  = titleId() → 'confirm.withdraw' → fillIn()
      │               "Withdraw KES 5,000\nto 0712345678?\nFee: KES 30"
      │    • body   = menu.body() → ["1. Confirm 2. Cancel"]
      │    • footer = none (nav:false)
      │    • join with \n, measure size, work out maxInput
      ▼
⑤ RENDERED  (what the "network" sends)
   {type:'CON', maxInput:1, encoding:{scheme:'GSM-7', units:66, limit:182},
    text:"Withdraw KES 5,000\nto 0712345678?\nFee: KES 30\n1. Confirm 2. Cancel"}
      │  phone.view(): add typed digits + soft-key labels
      ▼
⑥ VIEW  {kind:'session', text:"…", input:'', softkeys:{left:'Send', right:'Cancel'}}
      │  LcdDisplay.render(view)
      ▼
⑦ DOM   <div class="lcd" data-view="session"> … <p data-lcd="session-text">…</p>
      │  browser: CSS shows the session panel; pre-wrap turns \n into lines
      ▼
⑧ PIXELS  (160 × 128 px, VT323 font)
```

What ⑧ looks like:

```
┌──────────────────────┐
| Withdraw KES 5,000   |
| to 0712345678?       |
| Fee: KES 30          |
| 1. Confirm 2. Cancel |
| _                    |
| Send          Cancel |
└──────────────────────┘
```

Two different kinds of thing flow through this pipeline. Keep them apart in your head:

- **Rules** (①②③): the screen definitions. Written once, never change while the app runs, shared by every session.
- **State** (④): the session. Created when you dial, changed by every reply, thrown away when the session ends.

Rendering is just **rules + state → text**. That separation is what makes the design work.

2.2 Step ① — Writing the definition

File: `src/ussd/screens/payments.ts`

```
confirm: {
    type: 'menu',
    prompt: {by: 'service', cases: {withdraw: 'confirm.withdraw', airtime: 'confirm.airtime'}},
    options: {
        1: {label: 'confirm.yes', action: commitTransaction},
        2: {label: 'confirm.no', action: cancel},
    },
    inline: true,
    nav: false,
},
```

Read it field by field:

Field	Meaning in plain words
`type: 'menu'`	"This is a numbered menu with fixed options." It picks the *behaviour* in step ④
`prompt: {by: 'service', cases: …}`	"My title depends on the session's `service`: use `confirm.withdraw` for a withdrawal, `confirm.airtime` for airtime." One screen serves two flows
`options`	The numbered choices. `label` is a **message ID** (not text). `action` names a function to run that **ends** the session (the other choice, `next`, would go to another screen)
`inline: true`	Show the options on one line: `1. Confirm 2. Cancel` (saves a row on a 5-row screen)
`nav: false`	`0` and `00` do not work here. You must confirm or cancel; you cannot "go back" out of a money decision

What is checked right now, while you type in WebStorm: the object must match the `Screen` type in `ussd/types.ts`. Because of `type: 'menu'`, TypeScript knows it must be a `MenuScreen`, so it **requires** `options`, forbids `input`, and checks every `label` and `prompt` is a real message ID. A typo such as `label: 'confirm.yse'` is underlined red immediately. (Part 4 explains how: a *discriminated union*.)

What is *not* checked here: `next: 'bnakList'`. Screen IDs are plain strings, so a misspelt target compiles fine. That gap is closed later, in step ③. (Section 6 discusses how it could be closed at compile time instead.)

Why write screens as data at all? See section 6.1 for the full comparison. Short version: one object per screen is short, readable by non-programmers, and can be checked automatically. The alternative, one function per screen with `if` statements, is more flexible but repeats the same menu-drawing code everywhere and cannot be checked for length or translation as a whole.

Note what the definition contains: functions. `action: commitTransaction` is a reference to a function defined above it in the same file, and `validate: amountAllowed` on `enterAmount` likewise. The previous version of this project used *names* (`action: 'commitTransaction'`) looked up in separate registry files. Section 6.3 compares the two; both are valid.

2.3 Step ② — Compiling

`tsc` reads `payments.ts`, checks all the types, and writes `dist/ussd/screens/payments.js`. The confirm object there is **character-for-character the same, minus the type annotations**:

```
confirm: {
    type: 'menu',
    prompt: { by: 'service', cases: { withdraw: 'confirm.withdraw', airtime: 'confirm.airtime' } },
    options: {
        1: { label: 'confirm.yes', action: commitTransaction },
        2: { label: 'confirm.no', action: cancel },
    },
    inline: true,
    nav: false,
},
```

And `dist/ussd/types.js`, the compiled form of 225 lines of types, is just `export {};`. **Types do not exist when the app runs.** All the guarantees from step ① were enforced before the browser ever saw the code.

- **Why this step exists:** browsers run JavaScript, not TypeScript.
- **Necessary?** Yes, unless you drop TypeScript. (Modern Node can strip types itself, and bundlers like Vite do it on the fly, but some tool must.)

2.4 Step ③ — Joining features into one registry

File: `src/ussd/screens/index.ts`

```
export const registry: Registry = {
    entry: 'enterPin',
    home: 'main',
    screens: {...loginScreens, ...menuScreens, ...paymentScreens, ...statementScreens},
};
```

`...x` is *object spread*: copy every property of `x` into the new object. The result is one flat map from screen ID to definition:

```
screens = { enterPin, main, balance, withdraw, chooseNumber, enterPhone,
            bankList, enterBankAccount, enterAmount, confirm, miniStatement }
```

- `entry` = the first screen after dialling. `home` = where `00` goes, and reaching it clears the back history.
- **Why this step exists:** people want screens grouped by feature (one file per feature is easier to read), but the engine wants one lookup table (`registry.screens[id]`). This file is the bridge.
- **Necessary?** The grouping is a convenience; you could put all screens in one file. The single lookup table is needed.
- **Risk:** if two files used the same ID, the later spread would silently overwrite the earlier one. The IDs are unique today; nothing checks it. (Alternative: write a tiny `merge()` that throws on duplicates.)
- `SAMPLE_DATA` lives here too: three worst-case sessions (largest amounts, a 16-digit account) used by step ③b.

Step ③b — Startup checks: proving the data is sound before anyone sees it

File: `src/ussd/checks.ts` · called by `assertRegistry()` as the **first line** of `phone/main.ts`, and by the tests.

TypeScript checked the *shape* of each screen. It cannot check things that depend on *values*: does `next: 'bankList'` point to a screen that exists? Does the Swahili text of `confirm` fit in five rows when the amount is 70,000 and the account is 16 digits long? So, when the page loads, `checkRegistry()` runs three groups of checks and collects every problem as a sentence:

1. **Wiring.** `entry` and `home` exist; every screen's `type` is known; every `back` points to a real screen; then each screen type checks its own fields. For a menu, `SCREEN_TYPES.menu.check()` verifies each option key is digits only, is not a reserved key (`0`, `00`, `98`), and has **exactly one** of `next` or `action` with `next` pointing somewhere real.
2. **Translations.** Every English message ID exists in every other language, and every `{{x|formatter}}` names a formatter that exists.
3. **Lengths.** It builds a **fake session** for every combination of *language × sample data × screen × possible error*, renders it with the very same `renderSession()` used for real (step ④), and requires the result to fit 182/80 USSD units and the 23 × 5 grid, with no `{{` left. It does the same for every END message (23 × 6 grid). It pins the clock to **31 Dec 23:59** so dates are as wide as possible and uses a balance of **9,999,999.99**.

If anything fails, `assertRegistry()` throws one error listing every problem, and the emulator does not start.

- **Why it exists:** to turn "a Swahili translator made one string 3 characters too long" from a bug a member finds into an error a developer sees on page load.
- **Necessary?** Not for the app to *work*, but it is what makes "the menu is data" *safe*. Without it, data-driven menus are fragile.
- **Cost:** the check code (`checks.ts`) ships to every user and runs on every page load. It takes a few milliseconds. See 6.7 for running it only at build time instead.

2.5 Step ④ — The session: the state the rules are applied to

When you dial `*144#`, `engine.start()` creates a session. Every accepted reply changes it. Here it is, exactly as the engine holds it, at the moment we render the confirm screen:

```
{ "id": "s1",
  "screen": "confirm",
  "data": { "recipient": "0712345678", "service": "withdraw", "channel": "mobile", "fee": 30, "amount":
5000 },
  "history": ["main", "withdraw", "chooseNumber", "enterAmount", "confirm"],
  "page": 0, "open": true, "error": undefined,
  "startedAt": …, "lastActivityAt": …, "account": { "balanceCents": 1245000, … } }
```

How each field got its value (you will be asked this):

Field	Set by	When
`data.recipient = '0712345678'`	`engine.start()` default, and again by option *1. My Number* (`set: {recipient: config.myNumber}`)	Dial; then chooseNumber
`data.service / channel / fee`	Option *1. Mobile Money* on `withdraw`: `set: {service:'withdraw', channel:'mobile', fee:30}`	withdraw screen
`data.amount = 5000`	`enterAmount`'s input: `name: 'amount'`, `type: 'number'` turned `"5000"` into `5000`	enterAmount
`history`	`goTo()` pushes each screen; reaching `main` reset it to `['main']`	Every move
`screen = 'confirm'`	`enterAmount`'s `next: 'confirm'`	After 5000 was accepted
`error`	Cleared at the start of every reply; set only if a reply is rejected	Every reply

Notice: the session never stores **text**. It stores choices (`service`, `amount`) and the current screen ID. Text is produced fresh on every render, which is why switching language mid-session just works.

2.6 Step ⑤ — `renderSession()`: rules + state → text

Files: `ussd/engine.ts` (`render()` is one line that calls this) and `ussd/screenTypes.ts` (`renderSession()`). This is where the definition becomes text. It runs in this exact order.

⑤.1 Is the session closed?

```
if (!session.open) { const message = text(session.final ?? {id: 'end.cancelled'}); return {type: 'END', …}; }
```

A closed session shows only its END message (`end.success`, `end.cancelled`, `end.timeout`, `pin.locked`). Ours is open, so we continue.

⑤.2 Look up the rules and the behaviour

```
const screen = registry.screens['confirm'];        // the object from step ①
const type   = SCREEN_TYPES[screen.type];          // SCREEN_TYPES['menu']
const hasNav = screen.nav !== false;               // false
```

`SCREEN_TYPES` is a table with one entry per kind of screen: `menu`, `list`, `input`, `pages`, `info`. Each entry is an object with the same functions: `body()`, `reply()`, `maxInput()`, `check()`, `messages()`, and a flag `paged`. **This lookup replaces a `switch` statement.** Instead of "if menu do this, if input do that" scattered through the engine, the engine asks the table "how do I draw *this* type?". This is the **Strategy pattern**: each screen type is a strategy, picked at runtime by the `type` field. Section 6.2 compares this with a `switch`, including one real type-safety cost.

⑤.3 The title

```
const title = session.error ? text(session.error) : text({id: titleId(screen, session)});
```

No error, so we take the prompt. `titleId()` resolves it:

```
if (typeof prompt === 'string') return prompt;             // a plain message ID
const value = String(session.data[prompt.by]);              // session.data.service → 'withdraw'
return prompt.cases[value] ?? Object.values(prompt.cases)[0];  // → 'confirm.withdraw'
```

Intermediate value: the message ID `'confirm.withdraw'`, wrapped as a `Msg`: `{id: 'confirm.withdraw'}`.

What is a `Msg` and why not just a string? A `Msg` says *which* message and *with which values*, but not *in which language*. Keeping the meaning (`Msg`) separate from the words (text) is what allows: - re-rendering the same screen in Swahili at any moment; - logging language-independent IDs; - the startup checks to know every message a screen *could* show and measure each one.

It is necessary for the bilingual requirement. A simpler app with one language could use plain strings.

⑤.4 `Msg` → text: filling the placeholders

`text(msg)` is a small helper created at the top of `renderSession()`; it calls `messageText(msg, session, copy, now)` in `ussd/text.ts`, which calls `fillIn()`.

Step a — pick the template for the current language (`copy` is `COPY.en` or `COPY.sw` from `messages.ts`):

```
"Withdraw KES {{amount|kes}}\nto {{recipient|destination}}?\nFee: KES {{fee}}"
```

`\n` is a line break, placed by hand so no line exceeds 23 characters.

Step b — build the variables bag. `messageText()` merges the session's data with a few extra values every template may need:

```
{ ...session.data,                      // recipient, service, channel, fee, amount
  balance: 1245000,                     // from the account, in cents
  pinAttemptsLeft: 3, pinAttempts: 3,
  now: Date.now() }                     // the engine's clock, in milliseconds
```

Why one bag for every template? Simplicity: no template has to declare what it needs. The cost: any template can read any value, and a typo like `{{amout}}` is only caught by the startup check (it would be left as `{{amout}}` in the text, and the check looks for `{{`).

Step c — replace each placeholder. `fillIn()` runs a regular expression over the template:

```
const PLACEHOLDER = /\{\{(\w+)(?:\|(\w+))?}}/g;   // {{name}} or {{name|formatter}}
```

Placeholder	Variable value	Formatter	Result
`{{amount|kes}}`	`5000`	`kes`: thousands separator, Kenyan style	`5,000`
`{{recipient|destination}}`	`'0712345678'`	`destination`: if `channel` is `bank`, render the `dest.bank` message (bank name + A/C); otherwise the value as is	`0712345678`
`{{fee}}`	`30`	none	`30`

Intermediate value — the title text:

```
"Withdraw KES 5,000\nto 0712345678?\nFee: KES 30"
```

If a variable or formatter is unknown, `fillIn()` deliberately leaves the `{{…}}` in the text so the startup check can see it, instead of silently printing nothing.

Formatters keep words out of code. `destination` does not contain the words "A/C"; it renders another message (`dest.bank`), which exists in both languages. So formatters change *how a value looks*, never *which language* it is in.

⑤.5 The body

```
body = type.body(screen, session, text);   // SCREEN_TYPES.menu.body(...)
```

`menu.body()`:

```
const keys = Object.keys(screen.options).sort((a, b) => Number(a) - Number(b));   // ['1','2']
const choices = keys.map((key) => `${key}. ${text({id: screen.options[key].label})}`);
//            → ['1. Confirm', '2. Cancel']
return screen.inline ? [choices.join(' ')] : choices;     // inline → ['1. Confirm 2. Cancel']
```

Each option label is itself a `Msg` (`{id: 'confirm.yes'}`) turned into text by the same `text()` helper, so labels are translated the same way as titles. Keys are sorted as numbers so that `10` would come after `9`, not after `1`.

Intermediate value — the body: `["1. Confirm 2. Cancel"]`.

How the other screen types build their body (same function name, different strategy):

Type	`body()` returns	Example output
`menu`	one line per option (or one joined line if `inline`)	`1. Mobile Money`, `2. Bank Account`
`list`	items produced **at runtime** by `screen.items(session)`, numbered	`1. Pamoja Bank`, `2. Umoja Bank`, `3. Jamii Bank`
`input`	nothing: the title is the question	`[]`
`pages`	every line from `screen.lines(session)`; split into pages next	`11/11 Air -150`, …
`info`	nothing: the title is the content	`[]`

⑤.6 The footer, or pages

```
if (type.paged) { …paginate… } else { body = …; footer = hasNav ? '0. Back' : null; }
```

Our screen has `nav: false`, so **no footer**. Most screens get `0. Back` (translated: `0. Rudi`).

For a paged screen (only `miniStatement`), the body is split by `paginate()` and the footer is `98. More` `0. Back` on every page except the last. `paginate()` fills each page greedily with as many lines as fit **under every title the page could show** (the prompt, *Invalid choice.*, *Invalid input.*), so page breaks never move when an error replaces the title. With your demo account the 12 statement lines become 4 pages of 3:

```
page 1: "Mini Statement\n11/11 Air -150\n10/11 Dep 1,000\n09/11 Wdr -1,000\n98. More 0. Back"
page 2: "Mini Statement\n08/11 Air -50\n07/11 Dep 2,500\n06/11 Wdr -3,000\n98. More 0. Back"
…
```

⑤.7 Joining, measuring, input length

```
const lines    = footer === null ? [title, ...body] : [title, ...body, footer];
const fullText = lines.join('\n');
const maxInput = Math.max(type.maxInput(screen, session), hasNav ? KEYS.home.length : 0);
return {type: 'CON', text: fullText, encoding: measureUssd(fullText), maxInput};
```

- `fullText`: one string with `\n` between lines. That is what a real USSD gateway carries: one block of text.
- `measureUssd()`: counts GSM-7 units (66 here; limit 182). The phone can show this as a debugging counter.
- `maxInput`: how many digits the keypad will accept. A menu with options `1`–`2` allows 1 digit; if navigation were on, room for `00` (2 digits) would be added. This stops a user typing `12345` on a two-option menu.

⑤.8 The result: `Rendered`

```
{ "type": "CON",
  "text": "Withdraw KES 5,000\nto 0712345678?\nFee: KES 30\n1. Confirm 2. Cancel",
  "encoding": { "scheme": "GSM-7", "units": 66, "limit": 182 },
  "maxInput": 1 }
```

This is the **boundary between the network and the handset**. It contains only what a real gateway would send (CON/END and text) plus two handy measurements.

- **Why it exists:** so the phone never needs to know about screens, types, registries or messages. If the engine moved to a server, this is the shape the server would return.
- **Necessary?** Yes, as a boundary. Its exact fields are a choice; `encoding` exists only for the debug counter and the tests.

2.7 Step ⑥ — `phone.view()`: adding what only the handset knows

File: `src/phone/phone.ts`

The engine does not know what you have typed but not yet sent, or what the soft keys should say. The phone does. `view()` combines the phone's own state (its **mode**) with the `Rendered` result:

```
case 'session': {
    const screen = this.current();                // engine.render(session, lang) → step ⑤
    return { kind: 'session', text: screen.text, input: mode.input, encoding: screen.encoding,
             softkeys: mode.input ? copy.softkeys.sessionEditing : copy.softkeys.session };
}
```

Intermediate value — the View:

```
{ "kind": "session",
  "text": "Withdraw KES 5,000\nto 0712345678?\nFee: KES 30\n1. Confirm 2. Cancel",
  "input": "",
  "encoding": { … },
  "softkeys": { "left": "Send", "right": "Cancel" } }
```

Type `1` and the view becomes `input: "1"`, `softkeys: {left: 'Send', right: 'Clear'}` (the right key now deletes a digit instead of cancelling). **The engine is not called to type a digit** — only `view()` reruns. The engine is called again only when you press Send.

`View` has four kinds: `idle` (clock), `dial` (number being dialled), `session` (a CON screen + typed digits) and `message` (an END text, "Please wait…", an SMS, or the unknown-code error). It is a **discriminated union** (Part 4): each kind carries only the fields it needs.

- **Why a separate View and not just `Rendered`?** Because most of what the phone shows is not a USSD screen at all (clock, dialler, SMS, "Please wait…"), and even a USSD screen needs extra handset data. The View is the *complete* description of the LCD at one moment.
- **Necessary?** Yes, as long as you want the drawing code to be dumb. It also lets the tests check the screen without a browser: they pass a fake renderer that just stores each View.

2.8 Step ⑦ — `LcdDisplay.render()`: View → DOM

File: `src/phone/display.ts`

```
render(view: View): void {
    this.root.dataset.view = view.kind;                 // <div class="lcd" data-view="session">
    switch (view.kind) {
        case 'session':
            this.sessionText.textContent  = view.text;  // the screen text
            this.sessionInput.textContent = view.input; // the typed digits
            break;
        …
    }
    this.softLeft.textContent  = view.softkeys.left;    // "Send"
    this.softRight.textContent = view.softkeys.right;   // "Cancel"
    this.checkOverflow();
}
```

The constructor found every element once using `data-lcd="…"` selectors and kept references, so rendering is just a few property assignments.

- `textContent` (not `innerHTML`): the text is shown literally. Even if a template contained `<b>`, it would appear as the characters `<b>`. This is the XSS defence.
- This class is the only code that touches the LCD's HTML. Swap it for a canvas renderer, or a terminal printer, and nothing else changes.

2.9 Step ⑧ — The browser turns the DOM into pixels

Three CSS rules in `styles/emulator.css` finish the job:

1. **Only one panel is visible.** `.lcd[data-view="session"] [data-view-panel="session"] { … }` (and the same for `idle`, `dial`, `message`). Setting `data-view` in step ⑦ is all it takes to switch panels. No JavaScript hides or shows anything.
2. **`\n` becomes a line break.** `.lcd__text { white-space: pre-wrap; overflow-wrap: anywhere; }`. Normally HTML collapses newlines into spaces; `pre-wrap` keeps them *and* still wraps long lines. Without this one rule the whole screen would be on one line.
3. **The size is fixed.** `--lcd-width: 160px; --lcd-height: 128px;` and the **VT323** pixel font, sized so 23 characters fit a row.

Finally, `checkOverflow()` measures the real result: if the text box is taller or wider than its container (`scrollHeight > clientHeight`), it warns in the console and `phone/main.ts` shows a red banner. This is the last of **three** size defences:

Defence	When	How	Catches
Hand-placed `\n` in `messages.ts`	Writing copy	A person	Most problems
`lcdGrid` check in `checks.ts` (`displayRows`, `fitsDisplay`)	Page load	Simulates word-wrap on a 23-column grid	Every screen × language × worst-case data
`checkOverflow()` in `display.ts`	Every render	Measures the real DOM	Font or CSS surprises the simulation missed

2.10 The same pipeline with different inputs

Switching to Swahili changes only step ⑤.4 (a different `copy`). The session, the rules and every other step are identical:

```
"Toa KES 5,000\nkwa 0712345678?\nAda: KES 30\n1. Ndiyo 2. Hapana"     (GSM-7, 60 units)
```

After a bad reply (typing `20` on `enterAmount`), step ⑤.3 uses the error instead of the prompt. The error *replaces* the title, so the screen keeps the same height:

```
normal : "To: 0712345678\nEnter amount (KES):\n0. Back"
error  : "Invalid amount.\nEnter KES 50 to\n70,000.\n0. Back"
```

A `list` screen (`bankList`): step ⑤.5 calls `items(session)`, a function written next to the screen in `payments.ts`, to build the options at render time:

```
"Select bank:\n1. Pamoja Bank\n2. Umoja Bank\n3. Jamii Bank\n0. Back"
```

An `info` screen (`balance`): body is empty; the title template does the work:

```
template: "Account Balance\nKES {{balance|money}}\nAs at {{now|stamp}}"
text    : "Account Balance\nKES 12,450.00\nAs at 12/11 10:24\n0. Back"
```

A closed session: step ⑤.1 returns early with the END text, and the phone shows it with the `message` view and an `OK` soft key:

```
"Request received.\nYou will get an SMS\nconfirmation shortly.\nThank you."
```

2.11 Every intermediate value: why it exists, and could it be removed?

Value	Defined in	Why it exists	Could you remove it?
`Screen` (definition)	`types.ts`, `screens/*`	The menu as data	Only by writing each screen as code (section 6.1)
`Registry`	`screens/index.ts`	One lookup by ID; knows entry and home	No; the engine needs a lookup
`ScreenType` entry	`screenTypes.ts`	"How" for each kind of screen	Could become a `switch` (6.2)
`Session`	`types.ts`, created in `engine.ts`	The state per dial	No
`Msg`	`types.ts`	Meaning without language	Only with a single language
Variables bag	`text.ts` `messageText()`	Gives templates their values	Could pass only what each template needs; more typing, safer
Title / body / footer	`renderSession()`	A fixed layout every screen shares	Could let each type return the whole text; less consistent
`Rendered`	`types.ts`	Network → handset boundary	No, if the engine might move to a server
`Mode`	`phone.ts`	What the handset is doing	No
`View`	`phone.ts`	Complete description of the LCD	You could let the phone touch the DOM directly; harder to test
DOM text	`display.ts`	What the browser paints	No

Part 3 — The reply pipeline: from a key press to a new session state

Part 2 went from *state* to *pixels*. This part goes the other way: from your *finger* to a *changed state*. We are on `enterAmount` ("To: 0712345678 / Enter amount (KES):") and will type `5000`, press Send, and arrive on `confirm`. Then we look at what changes when the reply is rejected.

3.1 The whole journey

```
click "5" ─► phone/main.ts listener ─► phone.press('5')
              • next(): {session, input:''} → {session, input:'5'}
              • render() → view() → LcdDisplay        (engine state NOT changed)
… "0", "0", "0" the same way …                         input:'5000'

click Send ─► phone.press('soft-left')
              • next(): send key + input → engine.reply(session, '5000', 'en')   ◄ decided NOW
                  1. open? expired?
                  2. lastActivityAt = now; error = undefined
                  3. handle(): '' or '*'? → 00? → paged 98/0? → 0? → type.reply()
                       SCREEN_TYPES.input.reply() returns a Reply:
                       {selected:'5000', then:<enterAmount>, save:{amount:5000}}
                  4. apply: data.amount = 5000; proceed() → goTo('confirm')
                  5. record() → LogRecord → logRepo (not awaited)
                  6. return Outcome {status:'continue', transactional:false}
              • waitForNetwork(): setTimeout(deliver, 600 ms) → mode {pending}
              • render(): "Please wait..."

600 ms later ─► deliver(): open and not expired → mode {session, input:''}
                scheduleExpiry(); render() → Part 2 pipeline → confirm screen
```

3.2 Typing digits: only the phone changes

Files: `phone/main.ts`, `phone/phone.ts`

`phone/main.ts` attached a click listener to every `button[data-key]` at startup:

```
button.addEventListener('click', () => phone.press(key));   // key = button.dataset.key, e.g. '5'
```

Physical keyboard presses go through the same path: the `keydown` handler maps the key (`Enter` → `call`, `Escape` → `end`, `Backspace` → `soft-right`…), finds the matching on-screen button, and calls `button.click()`. **One code path for mouse and keyboard**, so they cannot behave differently.

`press()` is two lines:

```
press(key: PhoneKey): void {
    this.mode = this.next(this.mode, key);   // decide the new mode
    this.render();                           // show it
}
```

`next()` is the **handset state machine**. It looks at the current mode and the key. In `session` mode, a digit is appended if there is room:

```
if (/^\d$/.test(key)) {
    return mode.input.length < this.current().maxInput ? {...mode, input: mode.input + key} : mode;
}
```

Intermediate states after each key: `{kind:'session', input:'5'}` → `'50'` → `'500'` → `'5000'`. Each is a **new object** (`{...mode, input: …}`), the old one is not modified. A 7th digit would be ignored because `maxInput` for this screen is 6.

> **One thing to notice.** `this.current()` calls `engine.render()`, which rebuilds the entire screen text, only to read `maxInput`. It is cheap here (a few strings), so it does not matter, but it is not free. Alternative: remember the last `Rendered` in the phone when a reply is delivered. Pro: no repeated work. Con: one more piece of state that must be kept in sync (for example after a language switch). The current choice favours simplicity.

3.3 Pressing Send: the engine decides immediately

```
if (isSendKey(key) && mode.input) {
    return this.waitForNetwork(this.engine.reply(this.session!, mode.input, this.lang));
}
```

`engine.reply()` runs **now**, synchronously. The new session state is final before "Please wait..." even appears. The delay that follows is purely cosmetic: it imitates network latency.

Consequence you should be able to explain: if you confirm a withdrawal and then press End during "Please wait...", the money has already moved and the SMS still arrives. Real USSD behaves the same way: once the request has left the phone, ending the call does not undo it. A test covers exactly this.

3.4 Inside `engine.reply()`

File: `ussd/engine.ts`

```
reply(session, raw, lang) {
    if (!session.open) throw new Error('USSD session is not open');      // a bug, not a user error
    const expiry = this.expiry(session);
    if (expiry) return this.expire(session, expiry, lang);                // too late: time out
    session.lastActivityAt = this.now();
    session.error = undefined;                                             // forget the last error
    const from   = session.screen;                                        // 'enterAmount'
    const screen = this.screen(from);
    const input  = raw.trim();                                            // '5000'
    const forLog = SCREEN_TYPES[screen.type].secret?.(screen) ? REDACTED : input;
    const result = this.handle(session, screen, input, lang);
    this.record(session, from, forLog, result.selected ?? null, lang, result.status);
    return {status: result.status, transactional: result.transactional ?? false, notice:
result.notice};
}
```

Why each line is there:

Line	Why
Throw if closed	The phone should never send to a closed session. If it does, that is a programming bug, so fail loudly
Expiry check first	A real network would already have dropped the session; the reply must not be processed
Update `lastActivityAt`	Resets the 30-second inactivity clock
Clear `error`	Every reply starts fresh; a new error may be set below
`forLog`	The PIN screen's type says it is `secret`, so the log stores `****`, never the PIN
`handle()`	Works out what the input means and changes the session (next section)
`record()`	One log record per reply, whatever happened

3.5 `handle()`: what does the input mean?

The order of checks matters.

```
1. '' or contains '*'          → reject 'error.input'     (* would start a new USSD code)
2. nav on  and '00'            → goTo(home)                 continue
3. paged screen:
     '98' and not last page    → page + 1                   continue
     '0'  and not first page   → page − 1                   continue
4. nav on  and '0'             → goBack()                   continue
5. otherwise                   → SCREEN_TYPES[type].reply(screen, input, session)
```

Why navigation is checked before the screen's own reply: so that `0`, `00` and `98` mean the same thing on every screen. A side effect is that no menu can have an option numbered `0`, `00` or `98`, and the startup check forbids it.

Why the paged check sits between `00` and `0`: on the mini statement, `0` means "previous page" until you are on page 1, where it means "back" as usual.

Step 5: the screen type interprets the reply

For `enterAmount` (type `input`), `SCREEN_TYPES.input.reply()` runs:

```
const field = screen.input;
// field = {name:'amount', pattern:'^\\d{1,6}$', type:'number', validate:amountAllowed,
error:'error.input'}
if (!new RegExp(field.pattern).test(typed)) return {error: {id: field.error}};     // '5000' matches
const value    = field.type === 'number' ? Number(typed) : typed;                   // 5000 (a number)
const selected = field.secret ? REDACTED : typed;                                   // '5000'
const verdict  = field.validate ? field.validate.check(value, session) : null;    // amountAllowed →
null (OK)
if (verdict && 'end' in verdict) return {selected, end: verdict.end};
if (verdict) return {error: verdict};
const save = field.name && !field.secret ? {[field.name]: value} : undefined;     // {amount: 5000}
return {selected, then: screen, save};
```

`amountAllowed.check(5000, session)` (in `screens/payments.ts`) checks the limits for the service (withdraw: 50–70,000) and that `(5000 + 30) × 100 = 503,000` cents is not more than the balance of 1,245,000 cents. Both pass, so it returns `null`.

Intermediate value — the `Reply`:

```
{ selected: '5000', then: <the enterAmount screen, which has next:'confirm'>, save: { amount: 5000 } }
```

A `Reply` is one of three shapes (a discriminated union, told apart with `'error' in reply` and `'end' in reply`):

Shape	Meaning	Example
`{error: Msg}`	Not accepted; show this, stay	`{error: {id:'amount.invalid', vars: {min:50, max:70000}}}` for `20`
`{selected, then, save?}`	Accepted; save these values; go to `then.next` or run `then.action`	the one above
`{selected, end: Msg}`	A check ended the session	third wrong PIN → `{selected:'****', end: {id:'pin.locked'}}`

Why have a `Reply` at all? The screen type *decides*; the engine *acts*. `reply()` functions never modify the session; they only describe what should happen. All changes happen in one place, in `handle()`. That keeps the five screen types small and easy to test, and guarantees every change is logged the same way. (Strictly, validators *do* have one side effect: `checkPin` decrements `pinAttemptsLeft` on the account. See 6.9.)

Is it necessary? No. Each type could change the session itself. That would be shorter but would spread mutation across five places. Section 6.4 compares them.

Applying the Reply

```
if ('error' in reply) return reject(reply.error);           // session.error = …; status 'invalid'
if ('end' in reply)   { this.close(session, reply.end); return {status:'end', selected:
reply.selected}; }
if (reply.save) session.data = {...session.data, ...reply.save};      // data.amount = 5000
return this.proceed(session, reply.then, reply.selected);
```

`proceed()` follows `then`:

```
if (then.next) { this.goTo(session, then.next); return {status: session.open ? 'continue' : 'end',
selected}; }
const result = then.action!.run(session, this.now());       // e.g. commitTransaction
this.close(session, result.message);
return {status: 'end', selected, notice: result.notice, transactional: !!result.transactional};
```

`goTo('confirm')`:

```
const blocked = this.screen(id).guard?.check(session);   // confirm has no guard
if (blocked) { this.close(session, blocked); return; }
session.screen = 'confirm';
session.page = 0;
if (id === this.registry.home) session.history = [id]; else session.history.push(id);
```

Intermediate value — the internal `Result`: `{status: 'continue', selected: '5000'}`.

Session after the reply (only the changed fields):

```
screen : 'enterAmount' → 'confirm'
data   : {…, amount: 5000}
history: [main, withdraw, chooseNumber, enterAmount] → […, confirm]
lastActivityAt: now
```

3.6 Logging

`record()` builds one `LogRecord` and hands it to the log sink without waiting:

```
{ "sessionId": "s1", "phoneNumber": "0712345678", "screenId": "enterAmount",
  "timestamp": "2026-11-12T10:24:00.000Z", "input": "5000", "selectedValue": "5000",
  "nextScreenId": "confirm", "page": 0, "lang": "en", "status": "continue" }
```

`this.log.add(record).catch(…)`: the promise is not awaited. If writing to IndexedDB fails, a warning is printed and the session carries on. **Logging must never block or break the USSD flow.** Fire-and-forget is right for a training and debugging log like this one. For a real payment system's audit trail you would normally write the audit row in the same database transaction as the debit, so one can never exist without the other.

What each step of our walkthrough logged (real output, oldest first):

screenId	input	selectedValue	nextScreenId	status
null	`*144#`	null	enterPin	continue
enterPin	`****`	`****`	main	continue
main	`2`	`main.withdraw`	withdraw	continue
withdraw	`1`	`withdraw.mobile`	chooseNumber	continue
chooseNumber	`1`	`number.mine`	enterAmount	continue
enterAmount	`20`	null	enterAmount	**invalid**
enterAmount	`5000`	`5000`	confirm	continue
confirm	`1`	`confirm.yes`	null	**end**

3.7 Back on the phone: waiting, then delivering

`reply()` returned the **`Outcome`**: `{status: 'continue', transactional: false}`. The phone's `waitForNetwork()`:

```
this.clearTimer();
if (outcome.notice) this.scheduleSms(outcome.notice);           // only after a transaction
const delay = outcome.transactional ? config.transactionMs : config.responseMs;   // 600 ms here
this.timer = setTimeout(() => this.deliver(), delay);
return {kind: 'pending'};                                        // shows "Please wait..."
```

600 ms later, `deliver()`:

```
const expiry = session.open ? this.engine.expiry(session) : null;
if (expiry)            { this.engine.expire(…); this.mode = {kind:'message', source:{type:'ussd'}}; }
else if (!session.open)  this.mode = {kind:'message', source:{type:'ussd'}};   // show the END text
else                   { this.mode = {kind:'session', input:''}; this.scheduleExpiry(); }
this.render();                                                   // → Part 2 pipeline
```

`scheduleExpiry()` sets one timer for exactly `msUntilExpiry()` (the sooner of 30 s after the last reply or 180 s after dialling). If it fires and the session really has expired, the phone shows "Connection problem or invalid MMI code." whatever is on screen, as a real network would.

3.8 The three similar result objects

You will notice three objects that look alike. Here is why each exists:

Object	Produced by	Used by	Contains	Purpose
`Reply`	`SCREEN_TYPES[type].reply()`	`engine.handle()`	error, or selected + then + save, or selected + end	The screen type's decision, with no side effects

Object	Produced by	Used by	Contains	Purpose
`Result` (private)	`handle()` / `proceed()`	`reply()`	status, selected, notice, transactional	Everything needed for the log *and* the phone
`Outcome`	`reply()`, `cancel()`, `expire()`	`phone.ts`	status, notice, transactional, expiry	What the outside world is allowed to see

`Result` and `Outcome` differ only in that `Result` carries `selected` (needed for the log, not for the phone). They could be merged by adding `selected` to `Outcome`; the cost is that the phone would then see log-only data it should not care about. Keeping `Outcome` small keeps the engine's public contract small.

Part 4 — TypeScript, explained with this code

Every feature below appears in the project. Each comes with the line where it is used and, where useful, the Java or Go equivalent.

4.1 What TypeScript is (and is not)

- TypeScript = JavaScript + **types checked at compile time**. `tsc` checks them, then **deletes them**.
- At runtime there are no types. `dist/ussd/types.js` is literally `export {};`.
- So you cannot ask at runtime "is this a `MenuScreen`?" the way Java's `instanceof` works. You check **data** instead: `screen.type === 'menu'`.
- Typing is **structural**: if an object has the right fields, it *is* that type. No `implements` needed. Go interfaces work the same way; Java classes do not.

```
// tests/phone.test.mjs passes a plain object as the Renderer — no class, no "implements":
new FeaturePhone({render: (v) => views.push(v)}, account, 'en');
```

4.2 Declaring things: `const`, `let`, arrow functions

- `const` cannot be reassigned (like Java `final`); the object it points to can still change. `let` can be reassigned. `var` is never used.
- Arrow functions: `(a, b) => a + b`. Without braces, the expression is returned. They also keep `this` from the surrounding code, which is why `setTimeout(() => this.deliver(), delay)` works inside a class.

4.3 Annotations

```
export function measureUssd(text: string): Encoding { … }   // parameter type, return type
private timer?: ReturnType<typeof setTimeout>;              // optional field
```

Local variables are usually left for the compiler to **infer**.

4.4 `type` and `interface`

Both name a shape. The project uses `interface` for object shapes (`Session`, `Account`, `MenuScreen`) and `type` for unions and aliases (`Lang`, `Status`, `Screen`, `View`, `Prompt`).

Interfaces can **extend** one or more others:

```
interface ListScreen extends ScreenBase, Then { type: 'list'; items: ListSource; saveAs: keyof
SessionData; }
// = every field of ScreenBase (prompt, nav, back, guard) + Then (next, action) + its own
```

4.5 Literal types and unions (TypeScript's enums)

```
export type Lang   = 'en' | 'sw';
export type Status = 'continue' | 'end' | 'invalid' | 'cancelled' | 'timeout';
```

A **literal type** allows exactly one value; a **union** `A | B` means "one of these". Assigning `'finished'` to a `Status` is a compile error, yet at runtime it is a plain string, ideal for JSON and logs.

4.6 Optional fields, `undefined` and `null`

- `fee?: number` — may be missing (`undefined`).
- `screenId: string | null` — always present, but may be explicitly empty (the dial record has no screen).
- With `"strict": true`, the compiler forces you to deal with `undefined` / `null` before use.

4.7 Null-handling operators

Operator	Means	In the code
`a ?? b`	`a`, unless `null` / `undefined`, then `b`	`options.registry ?? defaultRegistry`
`a ??= b`	assign only if empty	`db ??= new Promise(…)` (open IndexedDB once)
`a?.b`	read only if `a` exists	`this.screen(id).guard?.check(session)`
`f?.(x)`	call only if `f` exists	`SCREEN_TYPES[screen.type].secret?.(screen)` — `info` screens have no `secret` function
`a!`	"trust me, not null" (no runtime check)	`this.session!`, `then.action!`

`??` is not `||`: `0 || 5` is `5`, but `0 ?? 5` is `0`. That matters for `fee: 0` on airtime.

Every `!` is a promise you make to the compiler. `then.action!` in `proceed()` is safe only because `checks.ts` guarantees each `Then` has exactly one of `next` / `action`, and the code tests `next` first.

4.8 Discriminated unions — the key to this codebase

A **discriminated union** is a union of object types that each have the same field with a different literal value. Checking that field tells the compiler which variant you have. Java's equivalent is a `sealed interface` with `record` implementations and pattern-matching `switch`.

The screens are one:

```
export type Screen = MenuScreen | ListScreen | InputScreen | PagesScreen | InfoScreen;
// each has   type: 'menu' | 'list' | 'input' | 'pages' | 'info'
```

So when you write `{type: 'menu', …}`, TypeScript knows it is a `MenuScreen` and requires `options`. Writing `{type: 'menu', input: {…}}` is an error. The previous version of the project inferred the kind from *which fields were present*, which needed a runtime check ("mixes options and input"); the `type` tag makes that mistake impossible to compile.

The phone's states and views are too:

```
type Mode =
    | { kind: 'idle' }
    | { kind: 'dialing'; value: string }
    | { kind: 'pending' }
    | { kind: 'session'; input: string }
    | { kind: 'message'; source: {type:'ussd'} | {type:'unknownCode'} | {type:'sms'; notice: Notice} };

switch (mode.kind) {
    case 'dialing': mode.value;     // OK — value exists only here
    case 'idle':    mode.value;     // compile error
}
```

And so is `Reply` in `screenTypes.ts` (`{error} | {selected, then, save?} | {selected, end}`), told apart with the `in` operator: `'error' in reply`.

`View` adds an **intersection** (`&`): `{softkeys: SoftkeyLabels} & (…union…)` = every view has soft keys, plus one variant's fields.

4.9 Narrowing and type guards

TypeScript narrows a union when you test it:

```
if (typeof prompt === 'string') return prompt;      // here prompt is a MessageId
const value = String(session.data[prompt.by]);       // here it is {by, cases}
```

A **type guard** is a function returning `x is T`:

```
const isDigitKey = (key: PhoneKey): key is DigitKey => (DIGIT_KEYS as readonly string[]).includes(key);
```

After `if (isDigitKey(key))`, `key` is a `DigitKey`.

4.10 Utility types

Type	Means	Where
`Partial<T>`	all fields optional	`set?: Partial<SessionData>` — an option sets *some* fields
`Readonly<T>`	fields cannot be reassigned	`Notice.data: Readonly<SessionData>`
`Record<K, V>`	typed map from `K` to `V`	`Record<Lang, Copy>`, `Record<string, Screen>`

Type	Means	Where
`ReturnType<F>`	what `F` returns	`ReturnType<typeof setTimeout>` (number in browsers, an object in Node)
`ReadonlyArray<T>`	array without `push`	`HISTORY` in `mockData.ts`

4.11 `keyof`, `typeof`, and indexed access

```
export type MessageId = keyof typeof EN_USSD;   // 'pin.prompt' | 'main.prompt' | … every key
```

`typeof EN_USSD` = the type of that object; `keyof` = the union of its keys. So the list of valid message IDs is *derived from the English text itself* and can never drift out of date. Every `prompt`, `label` and `error` in the screens is checked against it.

Indexed access reads a field's type:

```
export type DigitKey = (typeof DIGIT_KEYS)[number];              // '1' | '2' | … | '#'
export const SCREEN_TYPES: Record<Screen['type'], ScreenType<Screen>> = {menu, list, input, pages,
info};
//                                ^^^^^^^^^^^^^^ 'menu' | 'list' | 'input' | 'pages' | 'info'
```

Because the key type is `Screen['type']`, forgetting to add an entry for a new screen type is a compile error.

`keyof SessionData` limits names to real data fields: `saveAs: keyof SessionData` and `name?: keyof SessionData` — `saveAs: 'bnak'` will not compile.

4.12 `as const` and `satisfies`

- **`as const`** freezes the literal types: `['1','2'] as const` is `readonly ['1','2']`, not `string[]`. Used for `DIGIT_KEYS`, `KEYS`, `BANKS`, `AMOUNT_LIMITS`, `USSD_LIMITS`.
- **`satisfies T`** checks a value against `T` *without* changing its type to `T`.

```
const EN_USSD = { 'pin.prompt': '…', … } as const satisfies Record<string, string>;
```

`satisfies` makes sure every value is a string; `as const` keeps the exact keys so `MessageId` can be derived. A plain annotation `const EN_USSD: Record<string, string>` would make the keys just `string`, and every message ID would be accepted.

4.13 Generics

```
interface ScreenType<S extends Screen> {
    body(screen: S, session: Session, text: (msg: Msg) => string): string[];
    reply(screen: S, input: string, session: Session): Reply;
    …
}
const menu: ScreenType<MenuScreen> = { body: (screen) => … screen.options … };
```

`S` is a type parameter, `extends Screen` a constraint (as in Java). Inside `menu`, `screen` is a `MenuScreen`, so `screen.options` is known. Other generics: `querySelector<HTMLElement>(…)`, `required<T extends Element>(…)`, `createDataTable<Row>(…)`, `promised<T>(request: IDBRequest<T>)`.

4.14 A subtle one: method bivariance (why `SCREEN_TYPES` compiles)

`menu` is a `ScreenType<MenuScreen>`, yet it is stored in a table typed `ScreenType<Screen>`. Strictly, that is unsafe: the table promises "give me *any* screen", but `menu.body` only handles menus. It compiles because members written as **methods** (`body(screen: S): …`) are checked *bivariantly*, a deliberate TypeScript leniency. (Members written as properties with arrow types, `body: (screen: S) => …`, would be checked strictly under `strict` and this would fail.)

It is correct at runtime because the engine always looks up the entry with the screen's own `type` (`SCREEN_TYPES[screen.type]`). But the compiler is not proving that; the convention is. Section 6.2 discusses the alternatives.

4.15 Computed keys and assertions

```
const save = {[screen.saveAs]: index} as Partial<SessionData>;   // {bank: 0}
```

`[expr]: value` uses the *value* of `expr` as the property name. TypeScript cannot work out the exact type of such an object, so `as Partial<SessionData>` tells it. `as` is **not a runtime cast**: unlike Java, nothing is checked when the code runs. Other assertions: `button.dataset.key as PhoneKey`, `Object.keys(copies) as Lang[]`.

4.16 `unknown` instead of `any`

`any` switches type checking off; the project never uses it. `unknown` means "could be anything — check before use". `dataTable.ts` types untrusted input (saved settings, `setState` patches) as `unknown` and validates it with `Array.isArray`, `isRecord`, etc.

4.17 Classes

```
export class LcdDisplay implements Renderer {
    private readonly body: HTMLElement;
    constructor(
        private readonly root: HTMLElement,
        private readonly onOverflow: (o: boolean) => void = () => {},
    ) { … }
}
```

- `private readonly root` in the constructor is a **parameter property**: declares and assigns the field in one go.
- `private` is compile-time only; the field exists on the object at runtime.
- `implements Renderer` is optional (structural typing) but documents intent and gives earlier errors.
- `get language(): Lang` is a getter: read as `phone.language`.

4.18 Modules: `import`, `export`, `import type`, `.js`

```
import {config} from '../config.js';
import type {Session} from './types.js';
export type {Lang} from './ussd/types.js';        // re-export
```

- **Why `.js` when the file is `.ts`?** There is no bundler. The browser loads `dist/…/config.js` itself and needs the exact name. TypeScript maps `.js` to the `.ts` source while type-checking and leaves the path untouched in the output.
- **`import type`** imports only types and is erased. `types.ts` needs `Encoding` from `text.ts`, and `text.ts` needs `Msg` from `types.ts`; with `import type` on both sides, there is no runtime import cycle.

4.19 Asynchronous code

`Promise<T>` ≈ Java's `CompletableFuture<T>`. `async` functions return a promise; `await` waits for one.

```
const promised = <T>(request: IDBRequest<T>): Promise<T> =>
    new Promise((resolve, reject) => { request.onsuccess = () => resolve(request.result);
                                       request.onerror   = () => reject(request.error); });
```

That adapter turns IndexedDB's callback API into promises. `void load()` starts an async call on purpose without waiting. `this.log.add(record).catch(…)` is fire-and-forget.

4.20 Everyday syntax

Syntax	Means	Example
`{...a, ...b}`	copy `a`, then `b` over it	`session.data = {...session.data, ...reply.save}`
`[...a, x]`	array copy plus item	`[title, ...body, footer]`
`const {a = 0} = o`	destructuring with default	`const {amount = 0, fee = 0} = data`
`({account}) => …`	destructure a parameter	`check: ({account}) => …`
`` `${x}. ${y}` ``	string interpolation	`` `${key}. ${text(…)}` ``
`180_000`	digit separator	`config.ts`
`arr.at(-1)`	last element	`session.history.at(-1)`
`/^\d{4}$/`	regular expression	PIN pattern

4.21 What `tsconfig.json` switches on

`strict` (null safety and more), `noUnusedLocals` / `noUnusedParameters` (dead code is an error, which is why unused parameters are named `_session`), `noImplicitReturns` (every path returns). Details in 5.1.

Part 5 — File-by-file reference

Parts 2 and 3 followed the data. This part is the dictionary: every file, every exported name, every function. Read it with the file open.

5.0 What changed from the previous version

If you remember the first edition, these are the differences:

Before	Now	Effect
Flat `src/`	`src/ussd/`, `src/phone/`, `src/log/`	Folders match the layers
`screens.ts` (one file)	`ussd/screens/` with one file per feature, joined in `index.ts`	Each feature's screens sit next to the validators and actions they use
Screen kind inferred from which field is present	Explicit `type: 'menu' | 'list' | 'input' | 'pages' | 'info'`	Impossible mixtures cannot compile
Separate registries (`actions.ts`, `validators.ts`, `resolvers.ts`) referenced by **name**	Functions referenced **directly** in the screen (`action: commitTransaction`)	Fewer files and checks; screens are no longer plain JSON (see 6.3)
`engine.ts` (≈600 lines) did everything	`engine.ts` (sessions), `screenTypes.ts` (drawing + reply meaning), `checks.ts` (startup checks), `text.ts` (text)	Each file has one job
`encoding.ts`, `lcdGrid.ts`, `formatters.ts`	Merged into `ussd/text.ts`	All text helpers in one place
`index.ts`, `log.ts`	`phone/main.ts`, `log/main.ts`	`main.ts` = entry point of each page

5.1 Tooling and configuration files

`package.json`

```
"type": "module",
"scripts": { "build": "tsc && node scripts/build-site.mjs", "watch": "tsc --watch",
             "start": "tsc && node scripts/serve.mjs", "serve": "node scripts/serve.mjs",
             "test": "tsc && node --test \"tests/*.test.mjs\"" },
"dependencies": {}, "devDependencies": { "typescript": "^6.0.3" }
```

- **`"type": "module"`** makes Node treat `.js` files as ES modules (`import`/`export`) instead of CommonJS (`require`). The scripts and tests depend on this.
- **`dependencies: {}`** — nothing ships to the browser except your own code. This satisfies the brief's "zero-dependency" requirement.
- **`devDependencies`** — only TypeScript, needed to build, never at runtime. `^6.0.3` means "6.0.3 or any later 6.x".
- **`private: true`** stops accidental publishing to npm.

- `&&` in a script runs the second command only if the first succeeded. So a type error stops the build and the tests.

`tsconfig.json`

Setting	Value	Why
`target`	`ES2022`	Emit modern JS (classes, `??`, `?.`, top-level await) that current browsers run natively; nothing is down-levelled
`module`	`ESNext`	Keep `import` / `export` as-is so the browser loads modules natively
`moduleResolution`	`bundler`	Lets `./x.js` resolve to `./x.ts` during type-checking. (Named "bundler", but here there is no bundler; it is chosen for its permissive resolution rules)
`lib`	`ES2022`, `DOM`, `DOM.Iterable`	Type definitions for JS built-ins and browser APIs (`document`, `HTMLElement`, `indexedDB`, `NodeList` iteration)
`strict`	`true`	Turns on all strict checks, most importantly `strictNullChecks`
`noUnusedLocals` / `noUnusedParameters`	`true`	Unused variables are compile errors
`noImplicitReturns`	`true`	Every code path in a returning function must return
`forceConsistentCasingInFileNames`	`true`	`import './Engine.js'` fails if the file is `engine.ts` (matters on Linux, which is case-sensitive)
`skipLibCheck`	`true`	Do not type-check `.d.ts` files of libraries (faster)
`sourceMap`	`true`	Emit `.js.map` so browser DevTools show your `.ts` code when debugging
`rootDir` / `outDir`	`src` / `dist`	`src/ussd/engine.ts` → `dist/ussd/engine.js` (folders are kept)
`include`	`["src"]`	Only compile `src/`; tests are plain `.mjs` and are not compiled

`vercel.json`

```
{ "framework": null, "buildCommand": "yarn build", "outputDirectory": "public" }
```

Tells Vercel: no framework detection, run `yarn build`, and serve whatever is in `public/` as a static site. There is no server code in production; everything runs in the visitor's browser.

`.gitignore`

Ignores `node_modules/`, `dist/`, `public/`, log files, and editor folders. Generated output never goes into git.

`scripts/build-site.mjs`

```
const ROOT = resolve(import.meta.dirname, '..');
const OUT = join(ROOT, 'public');
const ENTRIES = ['index.html', 'log.html', 'styles', 'assets', 'dist'];
await rm(OUT, {recursive: true, force: true});
for (const entry of ENTRIES) await cp(join(ROOT, entry), join(OUT, entry), {recursive: true});
```

- `import.meta.dirname` is the folder of the current module file (Node 20.11+). `..` makes the project root.
- Deletes `public/` completely, then copies the five entries in. Deleting first guarantees no stale files survive from a previous build.
- **Top-level `await`**: allowed in ES modules, so there is no wrapping `async function main()`.
- Uses `node:fs/promises` (the Promise version of the file system API) — the `node:` prefix makes it explicit that this is a Node built-in.

`scripts/serve.mjs`

A tiny static file server for local development, built only on Node's `http` module (no Express).

```
createServer(async (req, res) => {
    const path = decodeURIComponent(new URL(req.url ?? '/', 'http://localhost').pathname);
    const file = normalize(join(ROOT, path.endsWith('/') ? `${path}index.html` : path));
    if (!file.startsWith(ROOT)) { res.writeHead(403).end(); return; }
    try { res.writeHead(200, {'Content-Type': TYPES[extname(file)] ?? 'application/octet-
stream'}).end(await readFile(file)); }
    catch { res.writeHead(404, {'Content-Type': 'text/plain'}).end('Not found'); }
}).listen(PORT, ...);
```

Step by step:

1. Parse the URL and decode `%20` and similar escapes.
2. A path ending in `/` serves that folder's `index.html`.
3. `normalize(join(...))` resolves any `..` segments.
4. **Path-traversal guard**: if the resolved file is outside the project root, return 403. This stops `GET /../../etc/passwd`.
5. Look up the MIME type by extension (`TYPES`). Correct MIME types matter: browsers refuse to run a module script served as `text/plain`.
6. `PORT` comes from the environment or defaults to `5173`.

It serves from the **project root** (not `public/`), so during development it uses `index.html`, `styles/` and `dist/` directly.

> **Gotcha after moving files.** `tsc` never deletes old output. When `src/engine.ts` moved to `src/ussd/engine.ts`, a stale `dist/engine.js` could survive and be copied into `public/` by `build-site.mjs`. Your `dist/` is clean at the moment, but the safe habit is to delete `dist/` before building after a restructure, or add a `"clean": "rm -rf dist public"` script.

5.2 Shared files at the top of `src/`

`src/config.ts`

One frozen object of tunable numbers. `Object.freeze` makes it read-only at runtime (and TypeScript treats it as read-only too); the nested `display` object is frozen separately because `freeze` is shallow.

Key	Value	Used by
`myNumber`	`0712345678`	"My Number", log records
`serviceCode`	`*144#`	The only code that opens a session
`pinAttempts`	3	`checkPin`
`sessionMs` / `inactivityMs`	180 s / 30 s	`engine.expiry()`
`responseMs` / `transactionMs`	600 ms / 1.5 s	Fake network delay in `phone.ts`
`smsDelayMs`	4 s	Confirmation SMS
`display`	23 columns, 5 session rows, 6 message rows	`text.ts` grids

`src/messages.ts`

All text, in both languages. Exports `SoftkeyLabels`, `MessageId`, `Copy`, `kes()`, `money()`, `COPY`, `formatIdleDate()`.

- **`EN_USSD`** — English USSD templates keyed by dotted IDs. `as const satisfies Record<string, string>` keeps the exact keys, so **`MessageId = keyof typeof EN_USSD`** is the list of valid IDs. English is the master list.
- **`Copy`** — everything language-specific: `softkeys` per phone state, `weekdays`, `months`, `unknownCode`, `pending`, **`sms(notice)`** (a function that builds the confirmation SMS) and `ussd` (the templates, typed `Record<MessageId, string>` so Swahili must have every key).
- **`kes(value, decimals = 0)`** — `70000` → `"70,000"` using `toLocaleString('en-KE')`.
- **`money(cents)`** — `1245000` → `"12,450.00"`.
- **`COPY = {en: EN, sw: SW}`**; **`formatIdleDate(lang, date)`** — e.g. `"Thu 12 Nov"` / `"Alhamisi 12 Nov"`.
- `main.deposit` exists in both languages but no screen uses it (the menu option is commented out).

`src/mockData.ts`

- **`BANKS`** — `['Pamoja Bank', 'Umoja Bank', 'Jamii Bank'] as const`. A chosen bank is stored by **index** (0–2).
- **`history(balanceCents, now)`** (private) — 12 dated transactions, one day apart, ending yesterday, calculated backwards so they end exactly on the current balance.
- **`createMockAccount(overrides = {}, now = Date.now())`** — balance KES 12,450.00 (`1_245_000` cents), PIN `1234`, 3 attempts. `overrides` (spread last) lets tests replace any field, e.g. `{pinAttemptsLeft: 0}`.
- Money is in **integer cents** because JavaScript numbers are floating point (`0.1 + 0.2 !== 0.3`), the same reason you use `BigDecimal` or integer cents in Java and PostgreSQL.

5.3 `src/ussd/types.ts` — every data shape

The file's own comment says it: *read this first; the rest of the code just moves these objects around.* No runtime code.

Group	Types
Account	`Service`, `Channel`, `TxnType`, `Transaction`, `Account`
Messages	`Vars` (a bag of values), `Msg` (`{id, vars?}`)
Session	`SessionData`, `Session`, `Status`, `Expiry`, `Notice`, `Outcome`, `Rendered`
Plug-ins	`Guard` (`check(session)` → end message or `null`), `Validator` (`check(value, session)` → `null` / `Msg` / `{end}`), `Action` (`run(session, now)` → END message + optional notice), `ListSource` (`(session) => Msg[]`)
Screens	`Prompt`, `Then` (`next?` / `action?`), `ScreenBase` (prompt, nav, back, guard), `MenuScreen` + `MenuOption`, `ListScreen`, `InputScreen`, `PagesScreen`, `InfoScreen`, **`Screen`** (the union), `Registry`
Log	`LogRecord`, `LogSink`

Every plug-in has a **`messages`** list: all the messages it *can* return. Functions cannot be inspected before they run, so this list is how `checks.ts` knows which errors and END texts to render and measure.

5.4 `src/ussd/screens/` — the menu as data

`index.ts`

Joins the feature files: `registry = {entry: 'enterPin', home: 'main', screens: {...loginScreens, ...menuScreens, ...paymentScreens, ...statementScreens}}`. Also exports **`SAMPLE_DATA`**, three worst-case sessions for the length checks (withdraw to mobile 70,000; withdraw to bank with a 16-digit account; airtime 10,000).

`login.ts`

- **`notLocked`** (Guard) — returns `pin.locked` if `pinAttemptsLeft <= 0`. Attached to `enterPin`, so a locked member is turned away right after dialling.

- **`checkPin`** (Validator) — right PIN: reset attempts to 3, return `null`. Wrong: decrement; at 0 → `{end: pin.locked}`; at 1 → `pin.wrongOne` ("1 attempt left"); else `pin.wrong` with `{left}`. A malformed PIN (not 4 digits) is rejected earlier by the pattern and costs no attempt.
- **`enterPin`** — `type: 'input'`, pattern `^\d{4}$`, `secret: true` (never saved or logged), `next: 'main'`, `nav: false`.

`menu.ts`

- **`main`** — `type: 'menu'`, options 1 Balance → `balance`; 2 Withdraw → `withdraw`; 3 Airtime → `chooseNumber` with `set: {service:'airtime', channel:'mobile', fee:0}`; 4 Statement → `miniStatement`. `nav: false` (you are already home).
- **`balance`** — `type: 'info'`; the prompt template shows balance and time.

`payments.ts`

- **`AMOUNT_LIMITS`** — withdraw 50–70,000; airtime 10–10,000.
- **`amountAllowed`** (Validator) — out of range → `amount.invalid` with `{min, max}`; `(amount + fee) × 100 > balanceCents` → `amount.noFunds`.
- **`banks`** (ListSource) — one `bank.option` message per bank.
- **`transactionRef(now)`** — e.g. `AKBEHKW0001`: `AKB` + last 5 base-36 characters of the time + a 3-digit counter (module-level, wraps at 1000).
- **`commitTransaction`** (Action) — debit `(amount + fee) × 100` cents, add a statement line, return `end.success`, `transactional: true` and a `Notice` with a **copy** of the session data (`{...data}`), so the SMS cannot change later.
- **`cancel`** (Action) — returns `end.cancelled`.
- **Screens:** `withdraw` (menu: mobile fee 30 / bank fee 100), `chooseNumber` (menu; title by `service`), `enterPhone` (input, `^0[17]\d{8}$`), `bankList` (list from `banks`, `saveAs: 'bank'`), `enterBankAccount` (input, 6–16 digits), `enterAmount` (input, number, `amountAllowed`), `confirm` (inline menu with the two actions, `nav: false`).

`statement.ts`

- **`statementLines`** (ListSource) — transactions newest first. `.slice().reverse()` copies before reversing because `reverse()` changes the array in place.
- **`miniStatement`** — `type: 'pages'`.

The full flow:

```
enterPin ──PIN ok──► main
main ─1─► balance                                        (info, 0. Back)
     ─2─► withdraw ─1 mobile─► chooseNumber
                   ─2 bank───► bankList ─► enterBankAccount ─► enterAmount
     ─3─► chooseNumber (airtime)
     ─4─► miniStatement                                  (pages: 98 more, 0 back)
chooseNumber ─1 My Number────► enterAmount
             ─2 Other Number─► enterPhone ─► enterAmount
enterAmount ─► confirm ─1─► commitTransaction (END)   ─2─► cancel (END)
```

5.5 `src/ussd/screenTypes.ts` — how each kind of screen behaves

Exports `KEYS`, `REDACTED`, `SCREEN_TYPES`, `renderSession()`, `titleId()`, `pagesOf()`, `pageFooter()`, `paginate()`.

- **`KEYS = {back: '0', home: '00', more: '98'}`** — work on every screen; no option may use them.
- **`REDACTED = '****'`** — logged instead of a secret reply.
- **`Reply`** (private) — what a reply means: `{error}`, `{selected, then, save?}`, or `{selected, end}` (Part 3).
- **`ScreenType<S>`** (private) — the contract each kind of screen fulfils:

Member	Job
`body(screen, session, text)`	Lines under the title
`paged`	Whether the body is split into pages
`reply(screen, input, session)`	What a reply means (navigation keys never reach it)
`maxInput(screen, session)`	Max digits (the engine adds room for `00`)
`secret?(screen)`	Must replies be hidden from logs?
`check?(screen, exists)`	Startup check of the screen's own fields
`messages?(screen)`	Errors it can show and END messages it can lead to, for the length checks

- **The five entries:**

Type	`body`	`reply`	`maxInput`
`menu`	sorted options, or one line if `inline`	look up the key; unknown → `error.choice`; else `{selected: label, then: option, save: option.set}`	longest key
`list`	`items(session)`, numbered	parse the number, check the item exists, save the **index** under `saveAs`; `selected` = item text in English	digits in the item count
`input`	none	pattern → convert → validator → save under `name` unless secret	`maxLength`
`pages`	`lines(session)` (split later)	always `error.choice` (only navigation keys work)	2 (`98`)
`info`	none	always `error.choice`	1

- **`checkThen(then, exists)`** — exactly one of `next` / `action`; `next` must exist.
  `actionMessages(thens)` — END messages of the actions involved.
- **`renderSession(session, registry, copy, now)`** — Part 2, step ⑤.
- **`titleId(screen, session)`** — resolves a plain or `{by, cases}` prompt.
- **`backLine(copy)`** — `"0. Back"`.
- **`pagesOf(session, screen, copy, now)`** — the body split into pages (one page if not paged), measured under the prompt and both error titles.
- **`pageFooter(copy, hasMore)`** — `"98. More 0. Back"` or `"0. Back"`.
- **`paginate(titles, lines, copy, grid)`** — greedy page filling; throws if a single line cannot fit on any page.

5.6 `src/ussd/text.ts` — words and sizes

- **`messageText(msg, session, copy, now)`** — builds the variables bag (session data + `balance`, `pinAttemptsLeft`, `pinAttempts`, `now`) and calls `fillIn`.
- **`formatters`** — `kes`, `money`, `bankName`, `destination`, `txnType`, `date` (`dd/mm`), `stamp` (`dd/mm hh:mm`). Formatters that need words (`destination`, `txnType`) render another message, so words always come from `messages.ts`.
- **`fillIn(copy, msg, vars)`** — replaces `{{name}}` / `{{name|formatter}}`; unknown names are left in place so checks can find them.
- **`formattersIn(template)`** — formatter names a template uses (for the startup check).
- **`measureUssd(text)`** — GSM-7 units (extended characters `^ { } \ [ ~ ] | €` count 2) with limit 182, or UCS-2 with limit 80 as soon as one character is outside GSM-7.
- **`fitsUssd(text)`**, **`DisplayGrid`**, **`SESSION_GRID`** (23 × 5), **`MESSAGE_GRID`** (23 × 6).
- **`displayRows(text, columns)`** — simulates word-wrap: splits on `\n`, then into "word + trailing spaces" tokens; wraps a word that does not fit the rest of the row; breaks words longer than a row; counts rows.
- **`fitsDisplay(text, grid)`**.

5.7 `src/ussd/engine.ts` — sessions

Exports `EngineOptions`, `UssdEngine`.

- **Constructor options** (all optional — **dependency injection** without a framework): `registry`, `copies`, `log`, `now` (clock), `newId`. Tests inject a fixed clock, predictable IDs and an in-memory log.
- **`render(session, lang)`** — calls `renderSession()`.
- **`pages(session, lang)`** — calls `pagesOf()`.
- **`start(account, lang)`** — cancels any open session (one phone, one session), creates a new one on `entry`, runs `goTo(entry)` (so the PIN screen's guard runs), logs the dial.
- **`cancel(session, lang)`** — open: close with `end.cancelled` and log `cancelled`. Already closed (e.g. after a confirmed transaction): only change what the handset shows; the transaction stands.
- **`expire(session, reason, lang)`** — close with `end.timeout`, log `timeout`.
- **`expiry(session)`** — `'duration'`, `'inactivity'` or `null`. **`msUntilExpiry(session)`** — time to the sooner deadline.
- **`reply(session, raw, lang)`**, **`handle()`**, **`proceed()`** — Part 3.
- **`goTo(session, id)`** — runs the target's guard; sets screen; resets page; home resets history, others are pushed.
- **`goBack(session)`** — pops; goes to the screen's fixed `back`, else the previous screen, else home.
- **`close(session, final)`** — `open = false`, set `final`, clear error, forget the active session.
- **`screen(id)`** — lookup that throws on unknown IDs.
- **`record(…)`** — builds the `LogRecord`; fire-and-forget.
- **`randomId()`** — `crypto.randomUUID()` (only on HTTPS or localhost), else a hand-made UUID v4.

5.8 `src/ussd/checks.ts` — startup checks

Exports `CheckOptions`, `assertRegistry()`, `checkRegistry()`.

- **`checkRegistry(options)`** — three stages, each run only if the previous found nothing: **wiring** (entry/home exist; known `type`; `back` exists; each type's own `check()`), **copy** (every English ID in every language; known formatters), **lengths**.
- **`screenMessages(screen)`** — the type's `messages()` plus the guard's messages.
- **`checkLengths(…)`** — a real `UssdEngine` with the clock pinned to 31 Dec 23:59 and a 9,999,999.99 balance renders every screen × language × sample × possible error, plus every END message, and checks USSD size, grid rows and leftover `{{`. Duplicates are removed with `[...new Set(problems)]`.
- **`assertRegistry()`** — throws one error listing all problems. Called first in `phone/main.ts` and in the tests.

5.9 `src/phone/phone.ts` — the handset

Exports `DIGIT_KEYS`, `DigitKey`, `PhoneKey`, `View`, `Renderer`, `FeaturePhone`.

Member	Job
`constructor(renderer, account, lang = 'en', now = () => new Date(), log?)`	Creates the engine with the phone's clock and the log; renders immediately
`language` (getter), `setLanguage(lang)`	Read / change language and re-render (session untouched)
`tick()`	Re-render when idle so the clock stays current (`main.ts` calls it every 10 s)
`press(key)`	`mode = next(mode, key)`; `render()` — the only input path
`next(mode, key)`	The state machine (below)
`current()`	`engine.render(session, lang)`
`cancel()`	Clear timer, cancel in the engine, show the END message
`waitForNetwork(outcome)`	Schedule SMS if there is a notice; start the 600 ms / 1.5 s timer; return `pending`
`deliver()`	When the timer fires: expired → time out; closed → show END; else back to `session` and `scheduleExpiry()`
`scheduleExpiry()`	One timer for `msUntilExpiry()`; re-arms if there was activity
`scheduleSms(notice)`	After 4 s, queue the SMS; show it now only if idle
`idleOrInbox()`	Next queued SMS, or idle
`clearTimer()`, `render()`, `view()`	Housekeeping; `view()` builds the `View` (Part 2, step ⑥)

State machine (`next()`):

```
idle      —digit———————————————► dialing
dialing   —digit (max 20) / clear / End———► dialing / idle
          —Send, value ≠ *144#———————————► message(unknownCode)
          —Send, value = *144#: engine.start► pending
pending   —Cancel / End———————————————————► message(ussd: "Session cancelled.")
          —timer: deliver()————————————————► session  or  message(ussd END)
session   —digit (up to maxInput) / clear———► session
          —Cancel (empty) / End————————————► message(ussd)
          —Send with input: engine.reply———► pending
message   —OK / End———————————————————————► next SMS in inbox, else idle
```

`Send` = Call, left soft key or OK.

5.10 `src/phone/display.ts` — the LCD

- **`required(root, selector)`** — `querySelector` that throws, so a broken HTML template fails at startup.
- **`LcdDisplay`** (implements `Renderer`) — constructor finds the nine `data-lcd` elements; **`render(view)`** sets `data-view`, fills the panel with `textContent`, toggles `is-long` (dialled number over 8 characters) and `is-centered` (cancelled), sets soft keys, then **`checkOverflow()`** measures the DOM and reports through `onOverflow`.

5.11 `src/phone/main.ts` — entry point of `index.html`

The **composition root**: the one place objects are created and connected.

1. `assertRegistry()` — refuse to start if the menu is broken.
2. Find the LCD and the overflow banner; create `LcdDisplay` with a callback that shows/hides the banner.
3. Wrap it in a `renderer` that also updates the optional USSD counter (the element is commented out in `index.html`, so this is dormant).
4. `new FeaturePhone(renderer, createMockAccount(), lang, undefined, createIndexedDbLogRepo())`.
5. Wire every `button[data-key]` click to `phone.press(key)`; keep buttons in a `Map`.
6. Language: `applyLang()` syncs radio buttons, the help panel, the phone, `<html lang>` and `localStorage`. `readStoredLang()` / `storeLang()` wrap `localStorage` in `try/catch` because storage can be unavailable.
7. Keyboard: ignore Alt/Ctrl/Meta and typing in inputs; map with `KEYBOARD_MAP`; add `is-pressed`; call `button.click()` (one path for mouse and keyboard); ignore auto-repeat.
8. `setInterval(() => phone.tick(), 10_000)`.

5.12 `src/log/logRepo.ts` — storing the session log

Exports: `LogFilter`, `LogRepo`, `createMemoryLogRepo()`, `createIndexedDbLogRepo()`.

`LogRepo extends LogSink` with `query(filter?)` (newest first) and `clear()`. Two implementations share the interface — the **Repository pattern**:

Implementation	Used by	Storage
`createMemoryLogRepo()`	tests (Node has no IndexedDB)	a private array
`createIndexedDbLogRepo(factory = indexedDB)`	the browser app and log page	the browser's IndexedDB database `akiba`, store `sessionLogs`

Both are **factory functions returning object literals** rather than classes. State lives in a **closure** (the `records` array, the `db` promise), which makes it truly private. This is a very common JavaScript idiom; in Java you would write a class with a private field.

`matches(r, f)` — a record passes if every provided filter matches: session ID substring, `timestamp >= from`, `timestamp <= to`, status equal. Comparing ISO-8601 strings with `>=` works because ISO timestamps sort lexicographically in time order.

`newestFirst(a, b)` — comparator using `localeCompare` on timestamps, descending.

Memory repo: `add` stores a copy (`{...record}`) so later changes to the caller's object cannot alter the log. `query` reverses (so same-millisecond records come out newest first, because `sort` is **stable** in modern JS) then filters and sorts. `clear` sets `length = 0`, emptying the array in place.

IndexedDB repo:

- IndexedDB is the browser's built-in transactional key-value database. Unlike `localStorage` (synchronous, strings only, ~5 MB), it is asynchronous, stores structured objects, supports indexes, and holds much more.
- **`promised(request)`** — adapts IndexedDB's callback API to a Promise: resolve on `onsuccess`, reject on `onerror`.
- **`open()`** — opens version 1 of the database **once** (`db ??= ...` caches the Promise, so concurrent callers share one open). `onupgradeneeded` runs only when the database is new or the version increases: it creates the store with an auto-increment `id` key and two indexes, `sessionId` and `timestamp`. This is IndexedDB's schema-migration hook — think of it as Flyway's `V1__init.sql`.
- **`store(mode)`** — opens a transaction on the store, `readonly` or `readwrite`, and returns the object store.
- **`add`** — inserts a copy.
- **`query`** — if the filter's session ID is a full UUID (regex check), use the `sessionId` **index** (like a `WHERE session_id = ?` that uses a B-tree index); otherwise load everything and filter in memory. Results are sorted newest first with the auto-increment `id` as a tiebreaker, and `id` is stripped before returning (`({id: _id, ...record}) => record`) so callers get the same shape as the memory repo.
- **`clear`** — empties the store.

`factory = indexedDB` as a default parameter allows injecting a fake IndexedDB (such as `fake-indexeddb`) in tests.

5.13 `src/log/main.ts` — the session-log page

Loaded by `log.html`. It shows every logged interaction in a sortable, filterable table, lets you replay one session, export CSV, and clear the log.

Constants: `CSV_FIELDS` (column order for export), `STATUSES`, and `Row = LogRecord & {id: string; step: number}` — an intersection type adding a row ID and the step number within its session.

Function	What it does
`required(selector)`	Throwing `querySelector`, same idea as in `phone/display.ts`
`readLang()`	Language from `localStorage`, defaulting to English
`label(value, lang)`	If `value` is a message ID (e.g. `main.balance`), show its text in the chosen language with newlines turned into spaces; otherwise show the value as-is. This is why the log stores IDs: the log page can show them in either language
`toCsv(records)`	Builds CSV text. `escape()` wraps a value in quotes if it contains `"`, `,` or a newline, doubling inner quotes — the RFC 4180 rules
`download(name, text)`	Creates a `Blob`, an object URL for it, and a temporary `<a download>`; clicks it; then revokes the URL to free memory
`withSteps(records)`	Records arrive newest first. Reverse to oldest first, count per session with a `Map` to assign `step` 1, 2, 3..., give each a string `id`, then reverse back
`pathOf(rows)`	For one session, the screen path: `dial → enterPin → main → balance → main → END`, skipping consecutive duplicates (an invalid input stays on the same screen)
`sessionFilter(state)`	Returns the session ID if the table is filtered to exactly one session (`sessionId eq …`) and there is no quick search; otherwise `null`
`init()`	Builds the page (below)

`init()`:

1. Find the page elements; read the language; set the dropdown.
2. `showPath(state)` — whenever the table state changes, if it is showing exactly one session, print that session's path above the table.
3. `createDataTable<Row>({...})` — the column definitions. Notable ones: `timestamp` as `datetime` in `en-KE`; `sessionId` shortened to 8 chars; `phoneNumber` and `page` hidden by default; `selectedValue` translated with `label()`; `status` rendered as a coloured badge built with `Object.assign(document.createElement('span'), {...})`. Default sort: newest first. `persistKey` remembers the view between visits.
4. **Row actions:** *Replay this session* sets the table state to "filter `sessionId = this`, sort by step ascending, page 1" — so replay is just a filter, no special code. *Copy session ID* uses the clipboard API (`?.` because it may be unavailable).
5. `load()` — query the repo, add steps, give the rows to the table, update the "N records in M sessions" counter (`new Set(...).size` counts unique sessions), refresh the path.
6. Event wiring: language change re-renders the table; Refresh reloads; **`visibilitychange`** reloads when you switch back to the tab (so new sessions from the emulator tab appear); Export downloads a dated CSV.
7. **Two-click clear**: the first click changes the label to "Click again to clear" and arms a 3-second timer; a second click within 3 s clears. This avoids the blocking `confirm()` dialog.
8. `await load()` for the first render. `void init()` at the bottom starts it without awaiting.

5.14 `src/log/dataTable.ts` — a reusable data-table component

The largest file (≈1,700 lines). The header comment says it is a **TypeScript port of the `dataTable.js` component from your Survey Pad project**, with identical behaviour, markup and class names. Only `log/main.ts` uses it. You do not need to know every line, but you should be able to explain its design and main flows.

What it does

A configuration-driven table with: column sorting (Shift+click for multi-column sort), a quick search bar, an advanced filter dialog (all/any conditions, typed operators), a column-visibility dialog, pagination, per-row action menus, view-state persistence in `localStorage`, and an optional **server mode** where data comes from an async function.

Public API

```
const table = createDataTable<Row>({container, label, columns, data, rowId, rowLabel, rowActions,
                                    pageSizes, pageSize, sorts, persistKey, emptyMessage,
                                    minHeight, maxHeight, onStateChange});
table.setRows(rows); table.refresh(); table.getState(); table.setState(patch); table.destroy();
```

`setState(patch)` is **atomic**: it validates every part first; if anything is invalid, nothing is applied and the errors are shown and returned as `{ok: false, errors}`.

Key types

Type	Meaning
`ColumnType`	`text`, `number`, `date`, `datetime`, `boolean`, `enum` — decides formatting, alignment, and available filter operators
`ColumnConfig<R>`	What the caller passes per column: `key`, `label`, `type`, `accessor`, `format`, `render`, `options`, widths, flags…
`Column<R>` (internal)	The normalised column with all defaults filled in
`Sort`, `Condition`, `Query`, `TableState`, `StatePatch`	Sort spec; one filter (`field`, `op`, `value`, `value2` for "between"); what server mode receives (includes an `AbortSignal`); the readable state; the untrusted patch (`unknown` fields)
`Operator`	`{id, label, arity, test}` — `arity` is how many values it needs: 0 (is empty), 1 (equals), 2 (between)
`Norm`	A normalised comparable value: `string | number | boolean | null`

Module-level helpers (outside the component)

Helper	Purpose
`el(tag, attrs, children)`	A tiny `createElement` helper (like a mini JSX). `class` → `className`; `on*` functions become event listeners; `true` → empty attribute; strings become text nodes. Typed with `HTMLElementTagNameMap` so `el('button')` is an `HTMLButtonElement`

Helper	Purpose
`icon(name, size)`	Inline SVG icons from the `ICONS` table
`toDate(raw)`	Parse to `Date`; `YYYY-MM-DD` is parsed as a *local* date to avoid the UTC-midnight off-by-one bug
`normalize(type, raw)`	Turn a raw cell into a comparable `Norm`: numbers, `YYYY-MM-DD` keys for dates, minute-precision timestamps for datetimes, booleans, else strings
`compareValues(a, b)`	Numeric subtraction, or an `Intl.Collator` with `numeric: true` so `"item 10"` sorts after `"item 9"`
`op(...)`, `OPERATORS`	The operator table per column type: text (contains, equals, starts with…), number/date (`=`, `>`, between…), boolean (is yes/no), enum (is/is not), plus is empty/not empty
`ANY`	A pseudo-column "Any text column" for the quick search
`operatorsFor`, `findOp`	Operators allowed for a column (respecting a column's `operators` whitelist)
`validDate`, `validateValue`, `validateCondition`	Validation. `validDate` rejects 30 Feb by round-tripping through `Date`. "Between" checks *From* is not after *To*
`sanitizeCondition`	Trim text and drop values an operator does not use
`select`, `iconButton`, `valueInput`, `valueControls`, `buildCondition`	Build the filter-row UI. `buildCondition` returns a `<div>` augmented with `validate()` and `hasBadInput()` methods via `Object.assign` (that is what `ConditionRow extends HTMLDivElement` types)
`openDialog`	Builds a native `<dialog>` with title, body, Reset/Cancel/Apply; `showModal()` gives focus-trapping and Escape for free; removed from the DOM on close

Inside `createDataTable` — the main flows

1. **Validate configuration** and throw early on mistakes (missing container, duplicate keys, enum without options, invalid initial sorts).
2. **Normalise columns**: defaults first, then the caller's config spread over them, then computed `type`, `align` and `options`.
3. **State** (`state`: page, size, sorts, filters, match, quick, hidden) and **view** (`view`: current rows, total, loading, error).
4. **`restore()`** reads saved state from `localStorage`, validates each piece, and keeps only the valid parts with a notice listing what was dropped. **`persist()`** saves it. Saved data is treated as untrusted.
5. **Data pipeline — `update()`**:
6. Client mode → **`computeClient()`**: build a test function per filter with `testFor()`, keep rows that pass the quick search and **all** (or **any**) filters, set the total, clamp the page, sort with a **decorate-sort-undecorate** step (compute each row's sort keys once, sort, strip; empty values always last), then slice the current page.
7. Server mode → **`fetchServer()`**: abort the previous request with `AbortController`, number the request (`reqSeq`), call the user's function, and **ignore the response if a newer request has started**. This prevents a slow old response from overwriting a newer one — a classic race-condition fix.
8. Then **`render()`**: `renderBusy`, `renderHead`, `renderBody`, `renderChips`, `renderFooter`, `renderBadges`.

9. `commit()` — after any user change: hide notices, persist, update, call `onStateChange(snapshot())`. `log/main.ts` uses that callback to show the session path.
10. **Sorting** — `toggleSort` cycles none → asc → desc → none; Shift adds a secondary sort, shown with a rank number.
11. **Panels** — `openFilters`, `openSorting`, `openColumns` each edit a **draft copy** and only write to `state` on Apply, so Cancel truly cancels. The columns panel will not let you hide the last visible column.
12. **Row menu** — a single shared popover element (`popover="auto"`, the native HTML Popover API), positioned next to the clicked ⋮ button, with arrow-key, Home/End and Tab keyboard handling. A `WeakMap` maps each button to its row without memory leaks.
13. **Accessibility** throughout: `aria-sort` on headers, `aria-live` on the range text, `role="alert"` on errors, `aria-invalid` / `aria-describedby` on bad filter inputs, a screen-reader-only `<caption>`.

A subtle point: `renderBody` and `openMenu` reference `api`, which is declared near the end of the function with `const`. That is fine because those functions only *run* later, after `api` exists. Calling one before the declaration line would throw a *ReferenceError* (the "temporal dead zone").

5.15 HTML pages

`index.html` — the emulator

- `<script src="dist/phone/main.js" type="module">` in the `<head>`. **Module scripts are deferred**, so they run after the HTML is parsed even though they are in the head.
- `<link rel="preload" as="font" ... VT323-Regular.ttf crossorigin>` — starts downloading the LCD font immediately, so the screen does not flash in a fallback font. (`crossorigin` is required on font preloads, otherwise the browser downloads the font twice.)
- **Toolbar**: title, link to the log page, EN/SW radio buttons in a `<fieldset>` with a visually hidden `<legend>` for screen readers.
- **Overflow banner** (`data-overflow-warning`, `hidden`, `role="alert"`).
- **Handset** (`<section aria-label="Feature phone">`):
- The LCD (`data-lcd="root"`, `aria-live="polite"` so screen readers announce new screen text), with a status bar (signal, title, battery — decorative, `aria-hidden`), four panels marked `data-view-panel="idle|dial|session|message"`, and the soft-key label row.
- The keypad: soft keys, Call, End, a navigation pad (only the centre OK is a real button; arrows are decorative), and the 12 digit keys with letter hints (ABC, DEF…). Every button has `data-key="..."`, which is all `phone/main.ts` needs.
- **Guide panels** (`data-guide="en"` and `"sw"`): how-to text in both languages; only one is visible.
- The USSD counter `<p data-ussd-counter>` is commented out.

`log.html` — the session log

Toolbar with count and back link; controls (language select, Refresh, Export CSV, Clear log); the path paragraph; and one empty `<div data-log-table>` (the script is `dist/log/main.js`) that `createDataTable` fills.

5.16 Stylesheets

`styles/emulator.css`

- `@font-face` for **VT323** (a pixel font that looks like a feature-phone LCD; `font-display: block` hides text briefly rather than showing the wrong font) and **Inter** (variable font for the page UI).
- **CSS custom properties** (variables) in `:root`: `--lcd-background: #c3cfa1` (the greenish monochrome LCD), `--lcd-ink`, casing and key colours, brand green, fonts, and crucially `--lcd-width: 160px; --lcd-height: 128px` — the brief's hard constraint lives in one place.
- Layout of the stage, handset casing, bezel, keypad grid, key press effect (`.is-pressed` and `:active`).
- **Panel switching by attribute**: rules of the form `.lcd[data-view="session"] [data-view-panel="session"] { display: … }` show only the active panel. This is why `display.ts` only sets `data-view`. The rule `.lcd__text { white-space: pre-wrap }` is what turns each `\n` into a visible line break.
- Utility classes: `.is-long`, `.is-centered`, `.visually-hidden`, `.overflow-warning`, the blinking input cursor.

`styles/data-table.css`

Styles for every `dt-*` class the data table generates (toolbar, chips, table, sticky header, pager, dialogs, popover menu, error states), carried over from Survey Pad. The scroll region uses the `--dt-max-height` / `--dt-min-height` variables set from the table options.

`styles/log.css`

Small page layout for the log, the path line, the danger button, and the coloured status badges `.log-status--continue|end|invalid|cancelled|timeout`.

5.17 Tests

Run with `yarn test`, which compiles first and then runs `node --test "tests/*.test.mjs"`. They use **Node's built-in test runner** (`node:test`) and `node:assert/strict` — no Jest, no Mocha, consistent with the zero-dependency goal. They import from `dist/`, so they test the exact JavaScript that ships.

`tests/engine.test.mjs` — the engine, without a phone

Setup helpers:

- `NOW` — a fixed timestamp; every engine gets `now: () => NOW`.
- `setup({account, now, lang})` — builds an engine with an in-memory log and sequential IDs (`s1`, `s2` …), starts a session, and returns helpers `reply(...inputs)` (send several replies in a row) and `text(lang)`.
- `login()` — `setup()` then reply `1234`.
- `fits(text)` — both USSD and display limits.

What is covered (grouped):

Area	Tests check that…
Registry	the real registry passes every startup check (imported from `dist/ussd/checks.js`)
Navigation	options resolve by key and merge `set`; `0` goes back and `00` home; `nav: false` blocks them on PIN, main and confirm; balance shows `0. Back` and is not an END
Input	invalid input keeps the screen and shows the error in place of the prompt; `*` is rejected everywhere; phone numbers must match the pattern; PIN digits are not masked on screen
Flows	mobile withdrawal deducts amount + KES 30 and returns a notice; bank withdrawal picks a bank, validates the account, charges KES 100; airtime has its own limits and no fee; cancelling the confirmation does not debit; "My Number" vs "Other Number"
PIN	malformed PINs are free, wrong ones count, three lock; a correct PIN resets the counter
Time	sessions time out after inactivity or total duration (by passing a mutable clock)
Language	switching keeps screen, data and history; every screen renders in both languages with no `{{` left
Pagination	pages fit both limits; every page but the last offers 98; lines keep order with none lost; pages are greedy; the footer is measured in its own language; the USSD limit is hard even when the display has room; page breaks do not move under an error; 98/0 move between pages; 98 on the last page is invalid; the page index is clamped after a language switch
Startup checks	they catch broken navigation and names, a missing Swahili message, a screen too long only in Swahili, a statement line that fits on no page
Logging	every interaction is logged with the gateway shape and the PIN is `****`; dialling again cancels the open session; the memory repo filters correctly, newest first

`tests/phone.test.mjs` — the handset, with mocked time

- `beforeEach` enables **`mock.timers`** for `setTimeout` and `Date`, starting at a fixed date; `afterEach` resets. `mock.timers.tick(ms)` moves fake time forward instantly, so a 30-second timeout test takes microseconds.
- The renderer is a plain object pushing each `View` into an array; `last()` returns the latest. **No DOM is needed** because `FeaturePhone` only talks to the `Renderer` interface.
- Helpers: `press`, `send` (type digits + call + wait the delay), `dial`, `dialAndLogin`.

Covered: a full Swahili balance session from idle to idle (including switching to English mid-session); session views carry encoding; the keypad stops at `maxInput`; statement paging with 98/0; a locked account is turned away right after dialling; the 30-second timeout; a confirmed withdrawal still delivers its SMS even if the session is then cut off; an SMS arriving mid-session waits until idle; unknown codes; cancel is logged as `cancelled`; SMS bodies fit one GSM-7 message in both languages.

> **Why test the compiled `dist/` JS instead of the `.ts`?** Node cannot run TypeScript directly without extra tooling (older versions), and testing the compiled output means you test exactly what the browser runs. The cost: you must compile first, which `yarn test` does.

Part 6 — Design choices, alternatives, weak spots, questions

Every design is a set of trade-offs. For each important decision below: what the code does, what else could have been done, and the pros and cons of each. The aim is that, when someone asks "why didn't you just…?", you can answer with reasons rather than "the AI wrote it that way".

6.1 Screens as data, versus other ways to build a menu

What the code does: each screen is an object; one generic engine interprets them all.

Option	How it would look	Pros	Cons
A. Screens as data (current)	`{type:'menu', prompt:'…', options:{…}}`	Short; readable by non-programmers; one engine draws every screen the same way; the whole menu can be checked automatically (wiring, translations, lengths); adding a screen needs no engine change	Behaviour that does not fit the five types needs a new type; logic is spread across `screens/`, `screenTypes.ts`, `engine.ts`; harder to follow one flow by reading top to bottom
B. One function per screen	`function confirm(session, input) { if (input === '1') … return 'Withdraw KES …' }`	Total flexibility; easy to read one screen; no indirection	Every screen re-implements numbering, Back, errors, length limits; no way to check all screens automatically; translations scattered; much more code
C. A big `switch` in the engine	`switch (session.screen) { case 'confirm': … }`	Everything in one file; simple to debug	Grows without limit; mixing rules for every screen; same checking problems as B
D. A state-machine library (e.g. XState)	Screens as states, replies as events	Visual tools, well-known model	A dependency (the brief says zero); learning curve; still needs custom drawing and length checks
E. Menu loaded as JSON from a server	Fetch `menu.json` at startup	Operations staff could change the menu without a deploy — a real goal for a USSD team	Needs functions referenced by name (see 6.3); needs runtime validation of untrusted JSON

Verdict to defend: A fits a USSD menu well, because USSD screens really are a handful of repeated shapes (menu, list, input, paged list, info). The brief also asks to "test menu depth" and "preview text constraints", which A makes automatic.

6.2 `SCREEN_TYPES` lookup table, versus `switch`, versus classes

What the code does: a table `{menu, list, input, pages, info}`; the engine calls `SCREEN_TYPES[screen.type].body(…)`.

Option	Pros	Cons
Lookup table of strategy objects (current)	All of one type's behaviour (`body`, `reply`, `maxInput`, `check`, `messages`) sits together; adding a type = one new entry; the engine has no `if (type === …)`	**Type safety relies on a convention**: the table is typed `ScreenType<Screen>` and only compiles because of method bivariance (4.14). Nothing stops `SCREEN_TYPES.menu.body(someInputScreen)` from compiling
`switch (screen.type)` in each function	TypeScript **narrows** `screen` inside each `case`, so it is fully type-safe; with `noImplicitReturns`, a missing case is a compile error	Each new type touches several `switch` statements (render, reply, maxInput, check, messages); one type's behaviour is spread across the file
A class per type (`class MenuType implements ScreenType<MenuScreen>`)	Familiar from Java; can share code through a base class	More ceremony; same bivariance issue when stored in one table; objects already do this job in JavaScript
Behaviour on the screen object itself (each screen carries its own `body()`)	No lookup at all	Screens stop being simple data; every menu repeats the menu logic

A way to keep the table and close the safety gap: a small generic helper that takes the screen and returns the matching entry, with the type check done in one place:

```
function typeOf<S extends Screen>(screen: S): ScreenType<S> {
    // one visible, documented assertion instead of an unseen bivariance hole
    return SCREEN_TYPES[screen.type] as ScreenType<S>;
}
```

This does not make it provably safe, but it puts the one unchecked step in a single, named place.

6.3 Functions inside screen definitions, versus names

What the code does now: `action: commitTransaction`, `validate: checkPin`, `guard: notLocked`, `items: banks` — direct references to functions in the same file.

What the previous version did: `action: 'commitTransaction'` — a string looked up in a separate `actions` registry, with `satisfies` deriving the valid names.

	Direct references (now)	Names (before)
Typo safety	Compiler checks the reference exists	Compiler checked the name against the registry's keys (also safe)
Reading	Ctrl-click jumps to the function; validator sits right above the screen that uses it	One extra hop to another file
Startup checks	No "unknown action" checks needed	Needed checks for unknown names

	Direct references (now)	Names (before)
Files	Fewer (`actions.ts`, `validators.ts`, `resolvers.ts` removed)	More
Serialisable (could the menu be JSON?)	**No** — JSON cannot hold functions	**Yes** — screens were plain data, so the menu could be stored in a database or edited by a tool
Reuse across features	Import the function where needed	Any screen could name any registered function

Verdict to defend: for a single-team emulator, direct references are simpler. If the SACCO wanted a non-developer to edit menus in a web tool or store them in PostgreSQL (option 6.1-E), names would be required again.

6.4 The `Reply` object, versus letting screen types change the session

What the code does: `reply()` returns a description (`{error}` / `{selected, then, save}` / `{selected, end}`); only the engine changes the session.

Option	Pros	Cons
Return a `Reply` (current)	Screen types are almost pure functions, easy to test; every change to the session happens in `handle()`, so logging, error clearing and closing are consistent; impossible for one type to forget to log	An extra object type to learn; a little more code
Let each type change the session directly	Fewer types; each type is self-contained	Five places that can change the session; easy to forget to clear the error or record the selection; harder to reason about

This is the same idea as the **command / event** pattern: compute a decision, then apply it in one place. In Java terms: the screen type is a validator returning a result object; the engine is the service that persists it.

6.5 A mutable session in the engine, versus immutable state in the phone

The two state machines use opposite styles, and it is worth knowing why.

	Engine `Session`	Phone `Mode`
Style	**Mutated in place** (`session.screen = id`, `session.history.push(id)`)	**Replaced** (`return {...mode, input: mode.input + key}`)
Why	One long-lived object referenced by the phone, the engine and the log; shaped like a server-side session record	Small, many transitions; a new object per key press makes `next()` behave like a pure reducer
Pros	Simple, fast, matches how you would update a session row	Old states are never corrupted; easy to compare before/after; very testable

	Engine `Session`	Phone `Mode`
Cons	Anyone holding the session can change it; harder to undo or compare states	Allocates a new object each key (irrelevant at this scale)

An immutable engine (`reply()` returning a *new* session) would make time-travel debugging and "what if" tests easy, at the cost of more copying and of the phone having to replace its session reference after every reply.

6.6 Screen IDs as plain strings

What the code does: `next: 'confirm'` is a `string`. A typo compiles and is caught by `checks.ts` at page load.

Option	Pros	Cons
Plain strings + startup check (current)	Feature files stay independent: `payments.ts` can point to a screen defined in another file without importing it	Typos are found at page load, not while typing
Typed IDs: `type ScreenId = keyof typeof allScreens` and `next?: ScreenId`	Typos underlined in the editor	Circular typing: the screen types would depend on the list of screens, which depends on the screen types; usually needs generics on `Registry` or a separate hand-written `ScreenId` union to maintain
A hand-written union `type ScreenId = 'main' | 'balance' | …`	Simple and safe	One more list to keep in sync (but the compiler would flag the mismatch)

Related risk: `{...loginScreens, ...paymentScreens}` silently keeps the **last** screen if two files use the same ID. A tiny `merge()` that throws on duplicates would close it.

6.7 Running the checks in the browser at startup

Option	Pros	Cons
At startup in the browser and in tests (current)	A broken menu can never reach a user, even if tests were skipped; the error appears immediately on page load	`checks.ts` is shipped and run for every visitor (milliseconds, a few KB)
Only in tests / CI	Nothing extra shipped	Relies on tests being run before deploy
As a build step (`node scripts/check.mjs` before `build-site`)	Fails the deploy instead of the page	One more script

For a training tool, failing loudly in the browser is a reasonable choice. For a public product you would move it to the build.

6.8 Text as one string with `\n`, and hand-placed line breaks

What the code does: the engine produces one string; `\n` separates lines; templates contain manual `\n` so lines are ≤ 23 characters; CSS `white-space: pre-wrap` shows them.

Option	Pros	Cons
One string (current)	Identical to what a real USSD gateway sends; measuring size is trivial	The display cannot style a line differently (e.g. bold title)
An array of lines, rendered as separate `<p>`	Styling per line; no `pre-wrap` needed	No longer the real USSD shape; measuring needs joining anyway
Let the LCD wrap automatically (no manual `\n`)	Simpler copy-writing	Real handsets wrap differently from browsers; you lose control of where lines break; the row simulation would be less accurate

6.9 Validators that change the account

`checkPin.check()` decrements `account.pinAttemptsLeft`. That is a **side effect inside something called a check**, which breaks the "screen types decide, engine acts" rule from 6.4.

- **Pro:** the PIN logic lives in one small function next to the PIN screen.
- **Con:** calling `check()` twice (for example in a future preview feature) would cost the user two attempts. Today this is safe only because nothing else calls it: the startup checks render screens but never call validators.
- **Alternative:** return the decision (`{wrongPin: true}`) and let an action or the engine update the counter. More plumbing, fewer surprises.

6.10 Where things run: browser, engine, phone

Choice	Pros	Cons
Everything in the browser (current)	No server; free hosting on Vercel; works offline	Balance and PIN can be edited in DevTools; not a security model (fine for training)
Engine on a server, phone in the browser	Same code shape a real gateway integration would use (the engine already returns CON/END + text)	Needs hosting, an API, persistence
Timers in the phone, not the engine	Engine stays synchronous and deterministic; tests need no fake timers for it	Expiry is checked in two places (engine on each reply, phone with a timer)

6.11 Framework-free, versus React

You know React, so expect this question.

	Plain TypeScript (current)	React + Vite
Brief	Satisfies "zero-dependency"	Violates it
Size / speed	Tiny; no build tool beyond `tsc`	Larger bundle, more tooling
UI code	Manual: `textContent`, `data-*` hooks, one `LcdDisplay` class	Declarative components; React re-renders from state
Fit	The UI is one fixed device with four panels; manual updates are easy	Pays off for many interactive screens and forms

Note how close the design already is to React's model: `phone.view()` computes a complete `View` from state, and `LcdDisplay.render(view)` updates the DOM to match. That is exactly "UI = f(state)".

6.12 Logging choices

- **IndexedDB vs `localStorage`:** IndexedDB is asynchronous, holds objects, has indexes and much more space. `localStorage` would block the page on every write and holds only strings.
- **Fire-and-forget:** the session never waits for the log. Right for a training log; wrong for a legal audit trail (see 3.6).
- **What is logged as `selectedValue`:** a message ID for menus (`main.withdraw`), **English text** for lists (`Pamoja Bank`, because `list.reply()` uses `COPY.en`), and the typed value for inputs. Consistent IDs or indexes would be cleaner; the log page copes by translating anything that looks like a message ID.

6.13 Known weak spots (raise them yourself)

1. **Client-side trust.** Balance, PIN and limits live in the page; anyone can change them in DevTools. Acceptable for an emulator; not a security model.
2. **No balance re-check at commit.** `amountAllowed` runs when the amount is typed; `commitTransaction` debits without checking again. Harmless with one user; in production re-check inside the debit transaction (`UPDATE … SET balance = balance - ? WHERE id = ? AND balance >= ?`).
3. **`SCREEN_TYPES` type hole** (4.14, 6.2).
4. **Screen ID typos only caught at startup; duplicate IDs silently overwrite** (6.6).
5. **`amountAllowed.messages` lists only the withdraw limits**, so the length check measures `amount.invalid` with "50 to 70,000" but never with the airtime "10 to 10,000". It works today because the withdraw text is longer.
6. **Unused messages are not reported** (`main.deposit` exists but no screen shows it).
7. **`checkPin` has a side effect** (6.9).
8. **`serve.mjs` path guard** uses `file.startsWith(ROOT)`; a sibling folder whose name starts with `akiba` could pass. Fix: compare against `ROOT + path.sep`. Local dev server only.
9. **The log is never pruned.**
10. **Stale `dist/` files** after moving sources (5.1).
11. **Mock history starts at −KES 3,000** before the first deposit (invisible in the statement).
12. **`phone.current()` rebuilds the whole screen to read `maxInput`** on every digit (cheap; noted for completeness).

6.14 Question bank

What does the project do, in one sentence? A browser emulator of a feature phone running Akiba SACCO's `*144#` USSD menu, in English and Swahili, used to test menus and text limits and to train staff, with every step logged.

Explain the pipeline from a screen definition to the LCD. The screen object (rules) and the session (state) go into `renderSession()`. It looks up the screen by ID and its behaviour by `type` in `SCREEN_TYPES`, builds a title (prompt or error, as a `Msg`, filled from the session through templates and formatters), a body from the type's `body()`, and a footer (`0. Back` or paging), joins them with `\n`, and returns a `Rendered` (CON/END + text + size + max input). The phone adds the typed digits and soft keys to make a `View`. `LcdDisplay` writes it into the DOM with `textContent`; CSS shows the right panel and `pre-wrap` turns `\n` into lines.

What happens when I press Send? The phone calls `engine.reply()` immediately. The engine checks it is open and not expired, clears the old error, handles `00` / `98` / `0`, then asks the screen type what the reply means. It applies that decision (error, end, or save + next/action), logs one record, and returns an `Outcome`. The phone shows "Please wait..." for 600 ms (1.5 s for a transaction), then renders the new state.

Why separate `screens/` (what) from `screenTypes.ts` (how)? So adding a screen is writing data, and adding a kind of screen is one table entry; the engine never changes. It also lets the startup checks treat every screen the same way.

Why is there a `type` field on screens? It is the discriminant of the `Screen` union. It lets TypeScript require the right fields for each kind, and lets the engine find the right behaviour in `SCREEN_TYPES`.

What is a `Msg` and why not plain strings? A message ID plus values, without language. It allows switching language mid-session, logging language-independent IDs, and checking every message a screen can show.

How does Swahili work? `COPY.sw` has the same IDs as `COPY.en`; the type `Record<MessageId, string>` forces every key. Rendering with `COPY.sw` instead of `COPY.en` is the only difference.

How do you guarantee text fits? Hand-placed line breaks; at startup, every screen × language × worst-case data × possible error is rendered and measured against the USSD limit (182 GSM-7 / 80 UCS-2) and a simulated 23 × 5 grid; at runtime, the DOM is measured and a banner appears on overflow.

How does pagination work? `paginate()` fills each page greedily so that title + lines + footer fits under every title the page might show; `98` = next, `0` = previous (or back on page 1).

Why can no option be `0`, `00` or `98`? The engine checks navigation keys before the screen's own reply, so such an option could never be chosen. The startup check forbids it.

If I confirm and then press End during "Please wait...", is the money returned? No. The engine committed when Send was pressed. Ending only changes the handset text; the SMS still arrives. Real USSD behaves the same.

Why is the PIN visible while typing? Real USSD shows digits. Protection is: the PIN is never stored in session data (`secret`) and is logged as `****`.

How are timeouts tested without waiting? The engine takes an injected clock; the phone tests use Node's `mock.timers` to move time instantly.

Why IndexedDB for the log? Asynchronous, structured, indexed, larger. `localStorage` is only for small settings.

What is method bivariance and where does it matter here? TypeScript checks method parameters loosely, so `ScreenType<MenuScreen>` can be stored as `ScreenType<Screen>`. It is why `SCREEN_TYPES` compiles; the code is correct because it always looks up by the screen's own `type`, but the compiler does not prove it.

Why do screens hold functions now instead of names? Shorter, Ctrl-clickable, fewer checks. The cost: the menu can no longer be stored as JSON. If menus had to be edited by a tool or loaded from a database, names would come back.

What would you change for production? Move the engine server-side behind the telco gateway; real accounts with hashed, rate-limited PINs in the core banking system; re-check balances inside the debit transaction with idempotency keys; write audit records transactionally; prune or archive logs; run the startup checks in CI.

Why no React? The brief asks for zero dependencies, and the UI is one fixed device. The design still follows React's idea: compute a complete `View` from state, then update the DOM to match.

6.15 Cheat sheet

File	One line
`config.ts`	Frozen numbers: code, timeouts, delays, grid
`messages.ts`	All text in EN + SW; `MessageId`; `kes` / `money`
`mockData.ts`	Demo account and banks
`ussd/types.ts`	Every data shape — read first
`ussd/screens/*`	The menu as data, with its validators and actions
`ussd/screenTypes.ts`	How each kind of screen draws and reads replies; `renderSession`; paging
`ussd/text.ts`	Templates, formatters, USSD size, row counting
`ussd/engine.ts`	Sessions: dial, reply, navigation, timeouts, logging
`ussd/checks.ts`	Startup checks: wiring, translations, lengths
`phone/phone.ts`	Handset state machine, fake delays, SMS inbox → `View`
`phone/display.ts`	`View` → LCD DOM; overflow check
`phone/main.ts`	Builds and wires the emulator page
`log/logRepo.ts`	Log storage (IndexedDB / memory)
`log/main.ts`	Log page: table, replay, path, CSV, clear
`log/dataTable.ts`	Reusable table from Survey Pad
`scripts/*.mjs`	Build the static site / serve locally
`tests/*.test.mjs`	46 tests on Node's built-in runner